

A Petri Net Semantics for BPMN Choreographies

Gabriele Genovese¹ , Cinzia Di Giusto¹ , Ivan Lanese² , and
Emilio Tuosto³ 

¹ Université Côte d’Azur, CNRS, I3S, France

² Olas Team, University of Bologna & Inria - Université Côte d’Azur, Italy

³ Gran Sasso Science Institute, Italy

Abstract. BPMN 2.0 is a widely used notation for documenting and executing business processes. The standard specification, written in natural language, may be ambiguous, motivating the need for a formal semantics. In this work, we formalise the semantics of BPMN Choreographies via Petri nets. We focus, in particular, on the OR-join gateway, whose activation condition depends on how many paths have been chosen at the corresponding OR-split gateway, that is, by nature, a condition that cannot be observed locally. As in other works, we exploit the distinction between dead and alive paths to reduce this global condition to a local one, a technique called Dead Path Elimination. We prove that our semantics is equivalent to existing ad-hoc approaches from the literature. A key advantage of our approach is that it enables the use of well-established techniques and tools for the analysis of Petri nets.

Keywords: BPMN Choreography · Petri Net · Semantics · OR-gateway

1 Introduction

Business Process Model and Notation (BPMN) [17] is a widely adopted standard for the specification of business processes. This notation yields an easy-to-learn graphical language featuring a rich set of constructs for describing activities, control-flow decisions, synchronisation points, and interactions between participants. For instance, a BPMN collaboration diagram describes the activities of a set of processes, as well as the interactions among them.

Besides process and collaboration diagrams, BPMN 2.0 introduced choreography diagrams [17, Chapter 11], which provides a more abstract description of the whole communicating system, focusing on the interactions among participants, without explicitly modelling their internal behaviour. The standard specification [17] is written in natural language, which may lead to ambiguities and interpretation issues.

This problem is particularly evident in the case of the *inclusive join gateway* (hereafter OR-join gateway). As described in the BPMN standard [17, page 352]:

“Inclusive Gateways are used for modeling points of synchronization of a number of branches, not all of which are active, after which one or

more alternative branches are chosen within a Choreography flow [...] and these are subsequently merged.”

The specification, however, does not clearly state the semantics of the “subsequently merged”. Hence, the behaviour of a BPMN choreography varies across different execution engines. Also, it is often the case that implementations do not faithfully comply with the BPMN specification [5, Section 2.4]; for instance, Camunda [1] contradicts the specification, while jBPM [22] only supports OR-split gateways closed by a corresponding OR-join.

Contributions. We propose an encoding of the OR-join and OR-split gateways, together with the standard BPMN components of BPMN choreographies, into classical Petri nets, grounded on the Dead Path Elimination (DPE) approach proposed in [24]. The encoding instantiate such technique through a simple static analysis of the syntactic structure of terms representing BPMN choreographies introduced in [5] to identify the portion of the Petri net capturing the “complementary” behaviour of choices. This enables the propagation of explicit signals on inactive paths so that join gateways can determine locally the resolution of each relevant path to compute the actual merge of the incoming branches.

Interestingly, the Petri net encoding of BPMN choreographies is essentially equivalent to the operational semantics given in [5] in terms of a labelled transition system. This is formally proved by the Theorems 1 and 2.

The motivation behind our approach lies in our long-term goal, which is to provide a formal foundation for the analysis and verification of BPMN choreographies. This paper moves the first steps towards such goal paving the way to the adoption of the rich ecosystems of tool-supported modelling and verification techniques around Petri nets.

Structure of the paper. We recall the basics of BPMN in Section 2 together with an overview of the semantics proposed in [5]. Our main contributions are in Sections 3 and 4; the former presents our encoding while the latter shows the relation between our encoding and the operational semantics in [5]. We discuss related work in Section 5 and Section 6 concludes with some final remarks and future directions.

2 Syntax and Semantics

In this paper, we consider only the BPMN elements depicted in Fig. 1a. The chosen elements capture the core control-flow behaviour and belong to the BPMN choreography part; however, apart from the task term, they can also be used for the corresponding elements in BPMN collaboration diagrams.

Example 1 below gives an intuition of how the graphical notation of BPMN can be used to specify a choreography.

Example 1. The BPMN diagram below specifies a reviewing process. We decorate the BPMN graphical notation with red labels e_n , which will be explained

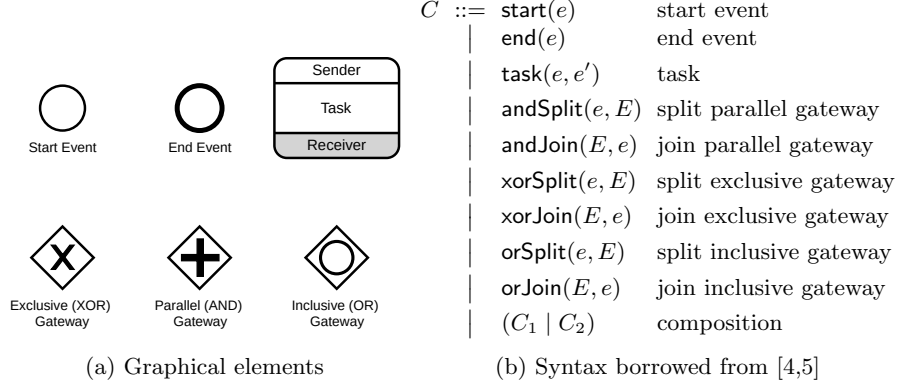
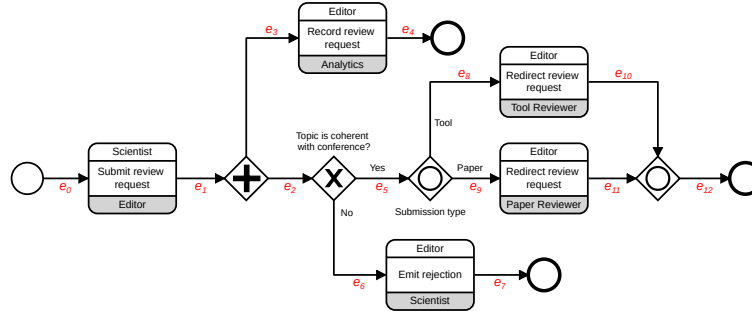


Fig. 1: Subset of considered BPMN elements



in Example 2. The submission of a review request, from a scientist to an editor, triggers two parallel activities (AND-split): one that terminates after the editor records the request for analytics purposes, and the other that examines the request. More precisely, the editor evaluates whether the topic is in the scope of the conference or not (XOR-split). An off-topic request is rejected, while for an on-topic one the process follows one or both of the two subsequent paths, depending on the submission type (OR-split). If the submission includes a paper, the editor redirects the request to a paper reviewer. If the submission includes a tool, the editor redirects the request to a tool reviewer. $\diamond$

To reason about BPMN choreographies, we follow the approach proposed in [4,5]. The idea of this approach is to represent BPMN choreographies with terms, dubbed *BPMN terms*, derived by the grammar in Fig. 1b. Such terms include start and end events, tasks, and three kinds of gateways: exclusive execution (XOR), parallel execution (AND), and inclusive execution (OR); finally, terms can be composed with the $\mid$ operator. We call *basic*, terms with no occurrences of the $\mid$ operator. Connections between basic terms are defined by

shared *edges* e , acting as references of *entry* and *exit points* of the control flow.⁴ We let E range over sets of edges. Each gateway comes in two variants: a split and a join. Split takes a single input and routes it towards a set of output edges; conversely, join synchronises multiple branches, having a set of input edges and a single output edge.

Briefly, $\text{start}(e)$ represents a BPMN event that produces a token on outgoing edge e ; analogously, $\text{end}(e)$ consumes a token from incoming edge e . A BPMN activity that consumes a token from e and produces one on e' is represented by the term $\text{task}(e, e')$. Gateway $\text{andSplit}(e, E)$ consumes one token from e and produces tokens on all edges in E . Its counterpart $\text{andJoin}(E, e)$ requires tokens on all the edges in E to produce one token on e . Gateway $\text{xorSplit}(e, E)$ consumes one token from e and produces one token on one (and only one) of the edges in E . Its counterpart $\text{xorJoin}(E, e)$ is activated when at least one edge in E contains one token and produces one token on e . Finally, $\text{orSplit}(e, E)$ consumes one token from e and produces a token on at least one edge in E . Conversely, $\text{orJoin}(E, e)$ synchronises a subset of incoming edges in E producing a token on e when no more tokens are expected from the remaining incoming branches. This condition is what makes the treatment of the $\text{orJoin}(E, e)$ gateway difficult: the activation of the rule depends on a condition on the “whole” choreography. The term $(C_1 \mid C_2)$ represents the composition of BPMN components C_1 and C_2 .

Example 2. The choreography in Example 1 can be represented by the term:

$$\begin{aligned}
 C_{\text{ex}} = & \text{start}(e_0) \mid \text{task}(e_0, e_1) \\
 & \mid \text{andSplit}(e_1, \{e_2, e_3\}) \\
 & \mid \text{task}(e_3, e_4) \mid \text{end}(e_4) \\
 & \mid \text{xorSplit}(e_2, \{e_5, e_6\}) \\
 & \mid \text{task}(e_6, e_7) \mid \text{end}(e_7) \\
 & \mid \text{orSplit}(e_5, \{e_8, e_9\}) \\
 & \mid \text{task}(e_8, e_{10}) \\
 & \mid \text{task}(e_9, e_{11}) \\
 & \mid \text{orJoin}(\{e_{10}, e_{11}\}, e_{12}) \mid \text{end}(e_{12}) \quad \diamond
 \end{aligned}$$

We say that a basic term C' is *one-step reachable from* C if the exit points of C are not disjoint from the entry points of C' ; *syntactic reachability* is the transitive closure of one-step reachability. For instance, in Example 2, $\text{task}(e_0, e_1)$ is one-step reachable from $\text{start}(e_0)$ and $\text{end}(e_4)$ is syntactically reachable from $\text{task}(e_0, e_1)$ (and hence from $\text{start}(e_0)$).

As in [4,5], a well-formed BPMN terms must satisfy some conditions. Structurally, one wants to ensure that no basic term appears in an isolated or unreachable portion of the model: every edge appears exactly once as entry point and exactly once as exit point; there is a unique occurrence of terms of the form $\text{start}(e)$; and at least one occurrence of terms of the form $\text{end}(e)$. Every basic term

⁴ We slightly simplify the notation of [4,5].

is syntactically reachable. Moreover, following [5], we require safety: i.e., no gateway can be activated twice. This entails that the BPMN on the right is unsafe: when the AND-split is activate, it produces two tokens that cause the XOR-join to fire twice.

From now on, we only consider BPMN terms satisfying the conditions above, which we call well-formed.

Moreover, to simplify the treatment, we consider acyclic BPMN choreographies, the generalisation to cyclic choreographies is discussed in the conclusion. BPMN specification also considers mixed gateways, which have multiple entry points and multiple exit points and combine split and join behaviour. We do not consider them since they can always be replaced by an equivalent pair of separate split and join gateways [14,3].

We adopt, as a reference semantics for BPMN, the *local semantics* presented in [5, Figures 11-13]. The key notion underlying the local semantics is that of a *marked process* C , which extends our term syntax by labelling each edge with a *status* highlighting its role in the control flow. More precisely, an edge $e.l$ is *live*, namely it is currently active; an edge $e.d$ is *dead* when it has been “deactivated”; an edge $e.w$ is in the *wait* status, representing the fact that the control flow has not reached e yet (cf. [5, Figure 10]). We use Σ to range over statuses and $\sqcup$ to denote disjoint union on sets of (possibly annotated) edges. We drop the status notation when we talk about the static structure of the BPMN term. The operational semantics is formalised as a labelled transition relation over marked BPMN terms, progressively updating edges’ status as the process executes: $C \xrightarrow{\alpha}_L C'$, where subscript L is used to distinguish this semantics from the Petri net one we will present later on. Label α represents the update operation on the edges’ status, needed to ensure all occurrences of the same edge are consistently updated.

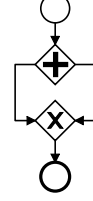
We recall some auxiliary notions from [5] to formalise the operational semantics. Given a set E of edges, functions $\text{setLive}(E)$ and $\text{setDead}(E)$ mark the edges in the set E as live and dead, respectively:

$$\text{setLive}(E) = \{e.l \mid e \in E\} \quad \text{and} \quad \text{setDead}(E) = \{e.d \mid e \in E\}$$

The predicate $\text{isLive}(E)$ holds when every edge in E is live (and likewise for $\text{isDead}(E)$), while $\text{isWait}(E)$ holds when at least one edge in E is in the wait status. Predicate $\text{isInit}(C)$ holds when all edges of C are in the wait state, capturing the initial configuration of the process before any step has been taken.

Fig. 2 reports a subset of the operational rules; the full semantics can be found in Section A. Indeed, we focus here on the rules for XOR and OR gateways, as they are representative of the semantics adopted by the framework and they showcase the main difficulties.

Rule **L-XorSplit-NC** models the execution of an XOR-split. When the incoming edge $e.l$ is live, exactly one outgoing edge e' is non-deterministically selected and marked as live, while all remaining outgoing edges (in E) are marked as dead. The label $\alpha = (d : \{e\} \sqcup E, l : e')$ highlights that edges in $\{e\} \sqcup E$ are turned dead and edge e' is turned live. Dually, **L-XorJoin-NC**, for an XOR-join,



$$\begin{array}{c}
\frac{E' = \{e'.1\} \sqcup \text{setDead}(E)}{\text{xorSplit}(e.1, \{e'.\Sigma\} \sqcup E) \xrightarrow{(\text{d}:\{e\} \sqcup E, 1:e')}_L \text{xorSplit}(e.d, E')} \quad (\text{L-XorSplit-NC}) \\
\\
\frac{E' = \{e'.d\} \sqcup \text{setDead}(E)}{\text{xorJoin}(\{e'.1\} \sqcup E, e.\Sigma) \xrightarrow{(\text{d}:\{e'\} \sqcup E, 1:e)}_L \text{xorJoin}(E', e.1)} \quad (\text{L-XorJoin-NC}) \\
\\
\frac{E_1 \neq \emptyset \quad E = \text{setLive}(E_1) \sqcup \text{setDead}(E_2)}{\text{orSplit}(e.1, E_1 \sqcup E_2) \xrightarrow{(\text{d}:\{e\} \sqcup E_2, 1:E_1)}_L \text{orSplit}(e.d, E)} \quad (\text{L-OrSplit-NC}) \\
\\
\frac{\neg \text{isWait}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} \neq \emptyset}{\text{orJoin}(E, e.\Sigma) \xrightarrow{(\text{d}:E, 1:e)}_L \text{orJoin}(\text{setDead}(E), e.1)} \quad (\text{L-OrJoin-NC}) \\
\\
\frac{\neg \text{isDead}(E), \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset}{\text{xorSplit}(e.d, E) \xrightarrow{\text{d}:E}_L \text{xorSplit}(e.d, \text{setDead}(E))} \quad (\text{D-XorSplit}) \\
\\
\frac{\text{isDead}(E)}{\text{xorJoin}(E, e.w) \xrightarrow{\text{d}:e}_L \text{xorJoin}(E, e.d)} \quad (\text{D-XorJoin})
\end{array}$$

Fig. 2: Selected rules from [5]

propagates a live status from an incoming edge $e'.1$ to the outgoing edge e , and marks all the other incoming edges as dead ($\text{setDead}(E)$). Rule **L-OrSplit-NC** models the execution of an OR-split. When the incoming edge $e.1$ is live, the edges of a non-empty subset E_1 of non-deterministically chosen edges of E are marked as live, while all remaining edges $E_2 = E \setminus E_1$ are marked as dead. Rule **L-OrJoin-NC** describes the behaviour of the OR-join. The gateway is activated when none of the incoming edges are in the waiting state ($\neg \text{isWait}(E)$), and at least one of them is live ($\{e'.\Sigma \in E \mid \Sigma = 1\} \neq \emptyset$). The conditions are satisfied through the DPE mechanism, which propagates dead statuses along the dead propagation branches and allows the OR-join to determine its enabling condition locally, as per the last two rules. Indeed, rules **D-XorSplit** and **D-XorJoin** ensure that once a branch status becomes dead, it is propagated through the process, enabling downstream gateways to correctly evaluate their activation conditions. In particular, each split gateway dead rule is activated when an incoming edge e is dead, all the outgoing edges are not dead ($\neg \text{isDead}(E)$), and no outgoing edges are live. This ensures the propagation of the dead status through $\text{setDead}(E)$. Dually, each join gateway dead rule is activated when all incoming edges E are dead and the outgoing edge is in the wait status. Then, the rule can set e with the dead status. Dead status propagation for split (resp. join) gateways occurs in the same way also for **andSplit**(e, E) and **orSplit**(e, E) (resp. **andJoin**(E, e) and **orJoin**(E, e)), whose rules are hence analogous (cf. Section A).

Not all marked terms are meaningful: we restrict to those that can be reached by executing the term from configurations where all edges are in the wait state.

Definition 1 (Reachable Marked Terms [5, Definition 2]). *A marked term C is reachable if there exists a marked term C' such that $\text{isInit}(C')$ holds and $C' \rightarrow_L^* C$, where $*$ denotes the reflexive and transitive closure.*

3 BPMN to Petri Net Encoding

We prove that the semantics presented in the previous section [5] can be faithfully simulated by Petri nets. As a result, BPMN models can be analysed using the large body of established Petri net theory and the numerous tools available for their verification. We start by recalling some notations and definitions:

Definition 2 (Petri Net). *A Petri net N is a triple (P, T, F) , where P is a finite set of places, T a finite set of transitions, and $F \subseteq (P \times T) \cup (T \times P)$ the flow relation. The preset and postset of a transition $t \in T$ are defined as: $\bullet t = \{p \in P \mid (p, t) \in F\}$ and $t \bullet = \{p \in P \mid (t, p) \in F\}$. A marking is a function $M : P \rightarrow \mathbb{N}$ assigning a number of tokens to each place. We normally consider Petri nets equipped with an initial marking M_0 . A transition t is enabled under a marking M if $M(p) \geq 1$ for all $p \in \bullet t$. The firing of a transition t enabled in a marking M produces the new marking*

$$M'(p) = \begin{cases} M(p) - 1 & \text{if } p \in \bullet t \setminus t \bullet \\ M(p) + 1 & \text{if } p \in t \bullet \setminus \bullet t \\ M(p) & \text{otherwise} \end{cases}$$

We write $(N, M) \rightarrow (N, M')$ when the firing⁵ of a transition yields M' from M .

Our encoding of BPMN is compositional, hence we need to define the union operator on Petri nets.

Definition 3 (Petri Net Union). *The union of two Petri nets $N = (P, T, F)$ and $N' = (P', T', F')$ is the Petri net $N \oplus N' = (P \cup P', T \cup T', F \cup F')$.*

While Corradini et al. [5] implement DPE at the level of an operational semantics by distinguishing between live and dead statuses, our approach embeds this distinction directly into the structure of the Petri-net encoding.

The key idea is to introduce a *dead propagation net*, namely a mirror copy of the relevant portion of the encoding in which each place and transition represents the dead counterpart of the corresponding element in the original net. The original net is used to propagate tokens representing a live status, while the mirror net propagates tokens representing a dead status. Consequently, gateways that may generate both live and dead outcomes, namely XOR-splits and OR-splits, are adapted to route tokens to the appropriate copy of the net.

The OR-join is then connected to both the original and the dead propagation nets. In this way, it can locally determine whether synchronization is complete: for each entry points, it receives a token either from the original net, corresponding to a live status, or from the dead propagation net, corresponding to a dead status.

Not all parts of the encoding require such a duplication. The dead propagation net is introduced only for the portion of the BPMN term that is syntactically

⁵ Since transitions are immaterial for us, we prefer a reduction semantics style for the firing relation instead of the standard Petri net notation.

reachable from an $\text{xorSplit}(e, E)$ or an $\text{orSplit}(e, E)$. Given a term C , we denote this set of edges by $\phi(C)$. We now introduce the preprocessing function that computes this set and identifies the part of the net that must be mirrored.

Example 3. The application of the function ϕ to the BPMN term C_{ex} in Example 2 is the set $\phi(C) = \{e_5, e_6, e_7, e_8, e_9, e_{10}, e_{11}, e_{12}\}$. $\diamond$

Now, let C be a reachable marked BPMN term (cf. Definition 1), and $E(C)$ its set of edges. For a set X , we define $\mathcal{P}_\emptyset(X) = \{Y \subseteq X \mid Y \neq \emptyset\}$.

The encoding $\widehat{C}$ of BPMN term C to Petri net N is:

$$\widehat{C} = \begin{cases} \widehat{C}_1 \oplus \widehat{C}_2 & \text{if } C = C_1 \mid C_2 \\ \text{PN}(C) \oplus \overline{\text{PN}}(C) & \text{if } C \text{ is a basic term and } E(C) \subseteq \phi(C) \\ \text{PN}(C) & \text{otherwise} \end{cases}$$

The cases of the definition of function $\text{PN}(\cdot)$ are given in Table 1 (which also includes a graphical representation of a simple instance of each case), while $\overline{\text{PN}}(\cdot)$ is given in Definition 5.

Encoding	Example
$\text{PN}(\text{start}(e)) = (P, T, F)$ where $P = \{e, p_s\}$ with p_s fresh $T = \{t\}$ $F = \{(p_s, t), (t, e)\}$	
$\text{PN}(\text{end}(e)) = (P, T, F)$ where $P = \{e\}$ $T = \{t\}$ $F = \{(e, t)\}$	
$\text{PN}(\text{task}(e, e')) = (P, T, F)$ where $P = \{e, e'\}$ $T = \{t\}$ $F = \{(e, t), (t, e')\}$	
$\text{PN}(\text{andSplit}(e, E)) = (P, T, F)$ where $P = \{e\} \cup E$ $T = \{t\}$ $F = \{(e, t)\} \cup \{(t, e')\}_{e' \in E}$	
$\text{PN}(\text{andJoin}(E, e)) = (P, T, F)$ where $P = E \cup \{e\}$ $T = \{t\}$ $F = \{(e', t)\}_{e' \in E} \cup \{(t, e)\}$	
$\text{PN}(\text{xorSplit}(e, E)) = (P, T, F)$ where $P = \{e\} \cup E \cup \{\bar{e} \mid e \in E\}$, $T = \{t_e \mid e \in E\}$, $F = \{(e, t_e), (t_e, e)\}_{e \in E} \cup \{(t_e, \bar{e}') \mid e, e' \in E, e' \neq e\}$	
$\text{PN}(\text{xorJoin}(E, e)) = (P, T, F)$ where $P = E \cup \{e\}$, $T = \{t_e \mid e \in E\}$, $F = \{(e, t_e), (t_e, e)\}_{e \in E}$	

$\text{PN}(\text{orSplit}(e, E)) = (P, T, F)$ where $P = \{e\} \cup E \cup \{\bar{e}' \mid e' \in E\}$ $T = \{t_S \mid S \in \mathcal{P}_\emptyset(E)\},$ $F = \{(e, t_S) \mid S \in \mathcal{P}_\emptyset(E)\}$ $\quad \cup \{(t_S, e') \mid S \in \mathcal{P}_\emptyset(E), e' \in S\}$ $\quad \cup \{(t_S, \bar{e}') \mid S \in \mathcal{P}_\emptyset(E), e' \in E \setminus S\}$	
$\text{PN}(\text{orJoin}(E, e)) = (P, T, F)$ where $P = \{e\} \cup E \cup \{\bar{e}' \mid e' \in E\}$ $T = \{t_S \mid S \in \mathcal{P}_\emptyset(E)\},$ $F = \{(t_S, e) \mid S \in \mathcal{P}_\emptyset(E)\}$ $\quad \cup \{(e', t_S) \mid S \in \mathcal{P}_\emptyset(E), e' \in S\}$ $\quad \cup \{(\bar{e}', t_S) \mid S \in \mathcal{P}_\emptyset(E), e' \in E \setminus S\}$	

Table 1: Petri net encoding

Definition 4 (Petri Nets Encoding). The encoding of a BPMN term C is $\llbracket C \rrbracket = (\hat{C}, M)$ with marking M such that $M(p_s) = 1$ if $\text{isInIt}(C)$ and

$$M(e) = \begin{cases} 1 & \text{if } e.1 \in E(C) \\ 0 & \text{otherwise} \end{cases}$$

$$M(\bar{e}) = \begin{cases} 1 & \text{if there are } (\bar{e}, t), (t, \bar{e}') \in F \text{ with } e.d, e'.w \in E(C) \\ 0 & \text{otherwise} \end{cases}$$

Intuitively, a live edge contributes a token on its corresponding place in the net while a dead edge contributes a token on its corresponding dead propagation place, provided that it is on a “frontier”, i.e., followed by an edge in the wait state. The set of such edges separates the portion of the BPMN process where propagation of dead notification already occurred from the one that has not yet been reached. Placing a token only at the frontier avoids flooding the dead propagation net with redundant tokens, and ensures that the OR-join can receive a dead token on each path corresponding to a non-activated original path.

The encoding is primarily inspired by the one in [9]. For start and end events, tasks, AND-split, and AND-join gateways the encoding is actually identical to the one in [9], barred for the end event, whose corresponding transition has in our case an empty postset. Let us focus on the encoding of the XOR-split and the OR gateways, for which our encoding differs from the one in [9].

XOR split. The encoding introduces one transition per exit point, all sharing the same input place. This models the non-deterministic choice: only one transition can fire, consuming the token in its preset. The fired transition, then, produces a token in the corresponding place, while simultaneously producing a token in each dead propagation place of the remaining exit point, representing the dead status that can be then propagated along the inactive branches.

XOR join. As in [9], the encoding introduces one transition per entry point, all sharing the same output place. This models a non-deterministic choice: each transition can fire independently when enabled, producing a token in the shared output place. The fact that exactly one entry point is active for a XOR-join is not forced here, but it follows from the safety condition: two active branches may result in two tokens in the shared place, violating safety. Note that one could use for the XOR-join an encoding which is the dual of the XOR-split. We have chosen this one for simplicity and to remain aligned, when possible, with [9]. The drawback is that some tokens which have no correspondence in BPMN semantics would remain in the dead propagation places, but as discussed in Example 5, we would have the same problem anyway, in more complex scenarios.

OR split. The encoding introduces one transition t_S for each non-empty subset S of E , representing the possible combinations of activated branches. All transitions share the same input place, modelling a non-deterministic choice. When transition t_S fires, it consumes the token in its preset and produces a token in all the places in its postset, namely in each place $e \in S$ and in each dead propagation place $\bar{e}$ for $e \in E \setminus S$, representing the dead statuses. Hence, for every choice made by the OR-split, there exists exactly one transition whose firing produces the corresponding marking.

OR join. The encoding introduces one transition for each non-empty subset S of E , representing every possible combination of active and inactive entry points. For a transition t_S to fire there must be a token in each $e \in S$ and one in each $\bar{e}$ for $e \in E \setminus S$. The firing of t_S consumes all such tokens and produces a single token in $t_S \bullet = \{e\}$. Therefore, the join is enabled only once every entry point has been resolved, either as live or dead, reducing the global activation condition of the OR-join to a purely local firing rule.

Remark 1. The exponential number of transitions in the size of E (the cardinality of $\mathcal{P}_\emptyset(E)$) in the encoding of OR-splits and OR-joins is not a practical limitation as the number of branches in BPMN models is generally small.

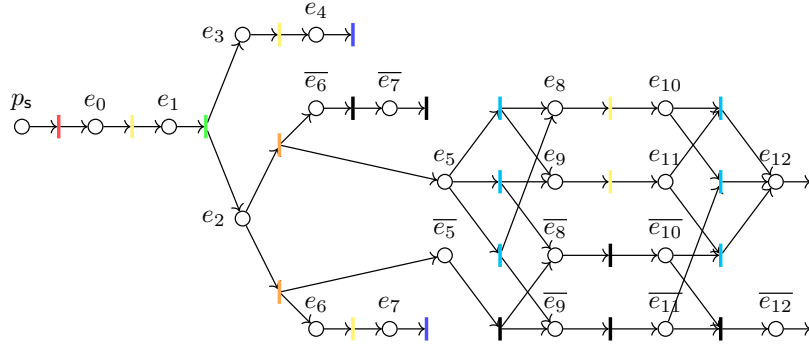
Finally, we discuss how the dead status can be propagated in our encoding. The dead status will have a correspondence in the *dead propagation net*. The dead propagation net has the same structure as the original one, but its places, transitions, and flow arcs are renamed (by adding an overline) w.r.t. the original ones. This is valid for every basic term except for XOR- and OR-gateways, whose dead propagation net differs. Given a set of edges E , we write $\bar{E}$ to denote the corresponding set of overlined edges, i.e., the set $\{\bar{e} \mid e \in E\}$.

Given that the dead propagation rules D-* for all the split and join-gateways share the same premises and the same effects on the edges' status, they must also behave the same in dead propagation net. Therefore, the encoding of XOR-gateways and OR-gateways match the one of the AND-gateways (w.r.t. the corresponding split and join rules).

Definition 5 (Dead Propagation Net). *Let C be a basic BPMN term. Its Dead Propagation Net $\overline{PN}(C)$ is the tuple $(\bar{P}, \bar{T}, \bar{F})$ where:*

- if $C = \text{xorSplit}(e, E) \vee \text{orSplit}(e, E)$, then $\bar{P} = \{\bar{e}\} \cup \bar{E}$, $\bar{T} = \{\bar{t}\}$, and $\bar{F} = \{(\bar{e}, \bar{t})\} \cup \{(\bar{t}, \bar{e}') \mid e' \in E\}$ with $\bar{t}$ fresh.
- if $C = \text{xorJoin}(E, e) \vee \text{orJoin}(E, e)$, then $\bar{P} = \bar{E} \cup \{\bar{e}\}$, $\bar{T} = \{\bar{t}\}$, and $\bar{F} = \{(\bar{e}', \bar{t}) \mid e' \in E\} \cup \{(\bar{t}, \bar{e})\}$ with $\bar{t}$ fresh;
- otherwise it is the overlined copy $\bar{P} = \{\bar{p} \mid p \in P\}$, $\{\bar{t} \mid t \in T\}$, and $\bar{F} = \{(\bar{x}, \bar{y}) \mid (x, y) \in F\}$ with $(P, T, F) = \text{PN}(C)$.

Example 4. The encoded version of the BPMN process in Example 3 is:



Transitions are colour-coded by component: red for the start event, blue for the end event, green for the AND-split, orange for the XOR-split, yellow for tasks, and cyan for the OR-split and OR-join. Black is used for every dead propagation transition, regardless of the component it belongs to.

The net starts with the fresh place p_s , introduced by the encoding of $\text{start}(e_0)$. From e_0 to e_4 , it contains the encoding of two tasks and an AND-split gateway. Since these edges do not belong to the scope of any XOR-split or OR-split, no dead propagation places or transitions are required. The AND-split is encoded by a transition producing tokens on both e_2 and e_3 , thereby enabling the two branches in parallel. One of them leads to the XOR-split starting from e_2 .

As shown in Example 3, all edges from e_5 to e_{12} belong to $\phi(C)$. Consequently, every component in this region is encoded into both its standard and dead propagation representations. The effect of this construction is particularly evident in the encoding of $\text{xorSplit}(e_2, e_5, e_6)$. When the transition selecting e_5 fires, it produces a token on e_5 and, at the same time, a token on $\bar{e}_6$. Thus, the choice of a branch immediately records the exclusion of the other one.

The next fragment contains the encoding of $\text{orSplit}(e_5, e_8, e_9)$ and the corresponding $\text{orJoin}(e_{10}, e_{11}, e_{12})$. Here, the powerset construction becomes explicit. The three cyan transitions connected to e_5 represent all admissible branch selections: one transition activates both e_8 and e_9 , while the other two activate a single branch and simultaneously place a token in the dead propagation place of the discarded one. In this way, the net records both the selected and the excluded paths. Dually, the OR-join is encoded by a family of transitions that can fire only when each entry point has been classified as either live or dead. The synchronization condition is therefore enforced directly by the net structure.

The two tasks between the OR-split and the OR-join also belong to the scope of the preceding XOR-split and are equipped with their dead propagation counterparts. For example, the transition following $\bar{e}_5$ propagates the dead status to both $\bar{e}_8$ and $\bar{e}_9$. These tokens are eventually consumed by the dead propagation part of the OR-join, whose transitions collect tokens from $\bar{e}_{10}$ and $\bar{e}_{11}$.

Finally, tokens are removed by the corresponding end-event encodings. Live tokens are consumed after e_4 and e_{12} , whereas dead tokens are discarded by the dead propagation net corresponding encoding connected to $\bar{e}_4$ and $\bar{e}_{12}$. $\diamond$

4 Correctness and Completeness

Having defined the encoding, we now establish that it faithfully preserves the operational semantics of marked BPMN terms.

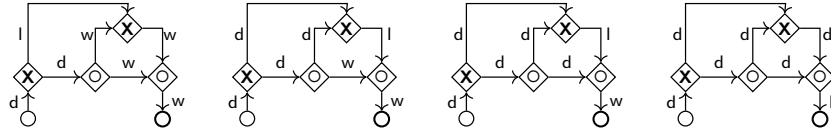
The correctness of the encoding intuitively boils down to have a corresponding firing for every local reduction step in the BPMN semantics; one would like to have the commuting diagram on the left below, but our encoding ensures a weaker property represented by the commuting diagram on the right:

$$\begin{array}{ccc}
 \begin{array}{ccc}
 & C \xrightarrow{\rightarrow_L} C' & \\
 \text{instead of } \downarrow \llbracket - \rrbracket & & \downarrow \llbracket - \rrbracket \\
 (N, M) & \xrightarrow{\rightarrow} & (N, M')
 \end{array}
 & \text{we get} &
 \begin{array}{ccc}
 & C \xrightarrow{\rightarrow_L} C' & \\
 \downarrow \llbracket - \rrbracket & & \downarrow \llbracket - \rrbracket \\
 (N, M) & \xrightarrow{\rightarrow} & (N, M')
 \end{array}
 \end{array}$$

$(N, M') \leq (N, M'')$

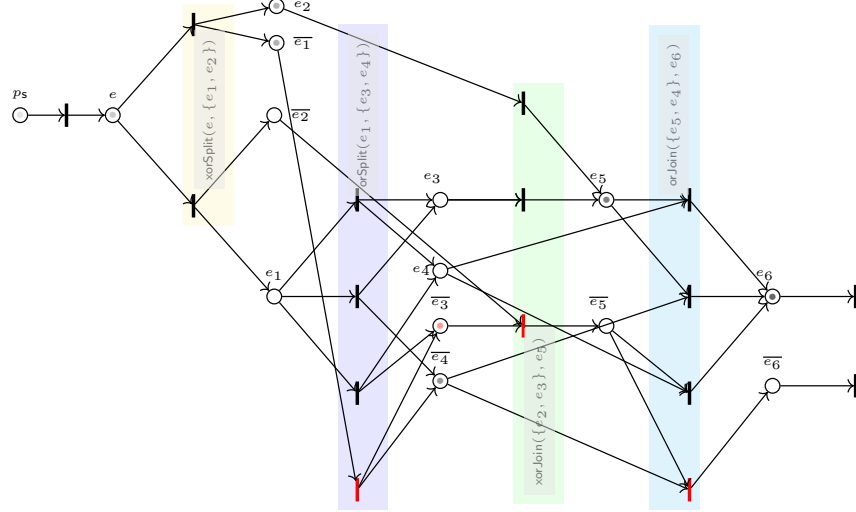
Essentially, a step $C \xrightarrow{\alpha} C'$ can in general be matched in the encoding $\llbracket C' \rrbracket = (N, M')$ “up-to” the presence of some tokens in the complement part of N . Such token may occur due e.g., to the way dead status propagation interacts with the encoding of XOR-split and OR-join gateways.

Example 5. Consider the diagrams below illustrating a dead path propagation



The transition from the first to the second diagram propagates the live status l through the XOR-split. The propagation of status d through the OR-split in

the second transition from the second to the third diagram is encoded as



where the red transitions are those belonging to the dead propagation net and the flow of tokens (from left to right) is represented by the intensity of the colour. The red token in place $\bar{e}_4$ is stuck in the dead propagation net since the transition in its postset is not enabled due to the fact that the XOR-split choose the branch on e_2 (hence, the place $\bar{e}_2$ is empty). $\diamond$

Theorem 1 accounts for this small discrepancy in the correspondence by stating that the complement portion of the encoding may possibly have more tokens than then in the net encoding the C' .

Theorem 1 (Correctness). *Let C be a reachable marked BPMN term, and $\llbracket C \rrbracket = (N, M)$ be its Petri net encoding and marking. If $C \xrightarrow{\alpha}_L C'$ then there is a firing $(N, M) \rightarrow (N, M')$ such that, assuming $\llbracket C' \rrbracket = (N, M'')$, $M'(e) = M''(e)$ for all $e \in E(C)$ and $M''(\bar{e}) \leq M'(\bar{e})$ for all $e \in \phi(C)$.*

Proof (sketch). The proof is by induction on the height of the derivation of $C \xrightarrow{\alpha}_L C'$, with a case analysis on the last applied rule. Safety is exploited to ensure that in case of live token propagation, outgoing edges initially have wait status. The detailed case analysis can be found in Section C. $\square$

For the completeness direction, we show that every firing step in the encoded net induces a BPMN reduction.

Theorem 2 (Completeness). *Let C be a reachable marked BPMN term, and $\llbracket C \rrbracket = (N, M)$ its Petri net encoding and marking. If $(N, M) \rightarrow (N, M'')$, then there exists a reachable marked BPMN term C' such that $C \xrightarrow{\alpha}_L C'$ and $(N, M') = \llbracket C' \rrbracket$, for all $e \in E(C)$, $M'(e) = M''(e)$ and for all $e \in \phi(C)$, $M''(\bar{e}) \geq M'(\bar{e})$.*

Proof (sketch). The proof is by structural induction on the BPMN term C . In each case, we show that every transition of $\widehat{C}$ has a corresponding transition from C . The detailed case analysis can be found in Section D.

Remark 2. The presence of unconsumed tokens shown in Example 5 does not spoil the completeness of our encoding. In fact, these additional tokens “move”, if at all, in the dead propagation net portion of the encoding without affecting the marking corresponding to the other portion.

Theorems 1 and 2 establish a full operational correspondence between the BPMN local semantics and our Petri net encoding, confirming that the encoding is both correct and complete with respect to the local reduction relation $\xrightarrow{L}$.

5 Related Work

Several formal semantics for BPMN 1.0 and BPMN 2.0 have been proposed in the literature, relying on a variety of target languages and formal models. The closest related work, as discussed above, is Corradini et al. [4,5], who introduce a direct operational semantics and study the relationship between choreography and collaboration diagrams. Their semantics provides a precise characterisation of BPMN behaviour and, in particular, offers one of the most comprehensive treatments of OR-join gateways. The main advantage of their approach is its direct correspondence with the BPMN standard. However, the resulting semantics is ad-hoc and cannot directly benefit from the extensive ecosystem of Petri net verification techniques and tools. Our work builds on their semantics and shows that it can be faithfully encoded into standard Petri nets, thus preserving its expressiveness while enabling automated analysis.

Dijkman et al. [9] model the OR-gateways through a combination of AND- and XOR-gateways following [8]. This solution is elegant and compact, requiring only standard Petri net constructs. However, it may introduce behaviours that are not valid BPMN executions. In particular, the resulting net may allow executions where none of the outgoing branches is selected, while BPMN requires at least one branch to be activated. Our encoding avoids this issue by explicitly representing every non-empty subset of selected branches. Although this results in an exponential number of transitions in the number of outgoing branches, BPMN models typically contain only a small number of alternatives, making the approach practical in realistic scenarios. Moreover, our encoding reduces the number of intermediate configurations. In the approach of Dijkman et al., tokens may temporarily remain inside the branch-selection mechanism while other parts of the process continue executing. This separation complicates the correspondence between BPMN executions and Petri net executions. In contrast, our encoding places tokens directly in their final destinations within a single transition firing. The resulting semantics is therefore more compact and simplifies the proof of operational correspondence.

A common solution to the OR-join problem is based on Dead Path Elimination (DPE) [13,24,10,2,20]. The key idea is to propagate information about

inactive branches so that an OR-join can determine locally whether all relevant paths have been resolved. Our work follows the same philosophy. However, rather than treating DPE as an additional operational mechanism, we integrate it directly into the structure of the Petri net through dead propagation places and transitions. This yields a uniform Petri net semantics in which dead-path information becomes an intrinsic part of the model.

Poizat and Salaün [19] translate BPMN choreographies into LOTOS NT and use the resulting specification to study choreography realisability (i.e., whether a BPMN collaboration behaves as its choreography counterpart dictates). Their framework also includes both OR-split and OR-join gateways. The main goal of the paper is to discuss the conformance between choreographies and their realisations, LOTOS NT is then used to check languages equivalence and no other kind of analysis are exploited. Our encoding enables the use of a wider set of analyses tools and techniques. More precisely, our Petri net models of BPMN processes are amenable of typical structural analysis done on Petri nets such as place invariants, liveness analysis, and reachability methods. Furthermore, our work establishes a formal correspondence with the BPMN semantics of [5], whereas such semantic preservation was not the focus of [19].

Other work encode BPMN into Coloured Petri Nets (CPNs) [21,15]. The main advantage of CPNs is their higher expressive power, which allows control-flow information to be represented directly through token colours and data values. However, the encodings proposed in [21,15] focus on the most commonly used BPMN constructs and do not provide a formal treatment of the OR-gateways. By contrast, our encoding relies only on standard Petri nets and supports the OR-split and OR-join gateways. This simplicity makes the resulting models compatible with a wider range of existing Petri net tools.

A number of proposals specifically address the OR-join problem through global reasoning techniques. Van Gorp and Dijkman [23] use graph transformations to formalise BPMN execution, while Kräuter et al. [12] rely on graph transformation systems for analysis purposes. Houhou et al. [11] provide a first-order logic formalisation of BPMN collaborations supporting formal verification. The main advantage of these approaches is their ability to express sophisticated global properties. However, the enabling condition of the OR-join is determined through a global analysis of the process state. Our semantics reduces the OR-join activation condition to a purely local Petri net firing condition. This locality is important both conceptually and operationally, as it avoids repeated global reasoning during execution and should result in a more efficient treatment.

Dechsupa et al. [6] exploit the additional expressiveness of CPNs to provide a local treatment of OR-join gateways. However, their framework is restricted to BPMN models where each split gateway is paired with a corresponding join gateway. We also impose restrictions of the well-formedness of BPMN terms but we slightly relax this requirement by allowing additional patterns, as the one in Example 5. Moreover, as underlined before, using standard Petri nets is more amenable to the use of a wider set of existing tools.

Beyond formal semantics, Napolitano [16] presents a tool for translating BPMN collaborations into Petri nets. However, the proposed encoding of the OR-gateways may generate unsafe Petri nets that duplicate tokens. Our encoding is instead designed to preserve the safety assumptions of the BPMN semantics and comes equipped with formal correctness and completeness results.

Finally, graphical choreographies have also been studied outside the BPMN setting. Deniérou and Yoshida [7] investigate choreographies in the context of multiparty session types, focusing on synthesis from communicating automata. More recently, Pal et al. [18] use graphical choreographies to model healthcare processes. These approaches share with BPMN choreographies the goal of describing global interaction protocols. However, their focus is on communication correctness and protocol synthesis rather than on the formalisation of BPMN semantics. Our work is complementary: by providing a Petri net semantics, it opens the possibility to connect BPMN-based modelling with verification techniques developed for communicating systems and behavioural types.

6 Conclusion and Future Works

We presented a Petri net encoding of BPMN choreographies and proved its equivalence with the operational semantics proposed by Corradini et al. [5]. The main contribution lies in the treatment of OR-split and OR-join gateways, whose semantics represents one of the most challenging aspects of BPMN. The resulting encoding provides a rigorous foundation for the analysis of BPMN choreographies through well-established Petri net techniques. This opens the possibility of investigating, in future work, the detection of deadlocks and the verification of type-theoretic properties such as protocol conformance.

This paper also establishes a well-grounded basis for extending the considered BPMN fragment towards additional BPMN collaboration constructs, incorporating, for example, send and receive events, event-based gateways, and message flows between pools and participants. These components are well documented in the literature, and their integration into our framework appears relatively direct.

An immediate future direction we will follow is the extension of our approach to handle a boarder class of BPMN terms. On the one hand we intend to support looping constructs. Extending the framework to cyclic models is particularly challenging. Our current encoding relies on the fact that dead-path information flows monotonically through the dead propagation net. In acyclic choreographies, once a dead token has been propagated along a branch, it can never influence the execution again. This property is crucial in the proofs of correctness and completeness, since extra tokens accumulating in dead propagation places remain harmless. In the presence of cycles, however, dead-path information may circulate indefinitely and interact with subsequent iterations of the process. As a consequence, dead tokens generated in one iteration may incorrectly affect the enabling conditions of gateways reached in later iterations. Therefore, extending the encoding to cyclic choreographies requires to distinguish different traver-

sals of a loop and to ensure that dead-path information is consumed or reset appropriately.

On the hand we plan to drop the safety assumption so to deal with BPMN terms where activities can be executed multiple times, e.g., as a consequence of an AND-split followed by a XOR-join.

Notice that cycles and (un)safety are two sides of a same coin; in fact, loops allow the execution of activities due to the reuse of tokens while unsafety uses multiple (possibly concurrent) tokens to the same effect.

Another future research direction is to revise the treatment of XOR-join gateways so that they consume both live and dead information, symmetrically to the way XOR-split gateways generate it. Such a modification could prevent the accumulation of obsolete dead tokens while preserving the local nature of gateway activation. Investigating these alternatives and their correspondence with the BPMN standard is an interesting topic for future work.

Finally, we are currently develop a tool implementing the proposed encoding. We aim at integrating this tool with existing Petri net frameworks for verification of system in order to verify BPMN choreographies.

References

1. Camunda services GmbH: Camunda v.7.14.0. <https://www.camunda.com> (2026)
2. Christiansen, D.R., Carbone, M., Hildebrandt, T.: Formal semantics and implementation of BPMN 2.0 inclusive gateways. In: International Workshop on Web Services and Formal Methods. pp. 146–160. Springer (2010)
3. Corradini, F., Ferrari, A., Fornari, F., Gnesi, S., Polini, A., Re, B., Spagnolo, G.O.: A guidelines framework for understandable BPMN models. *Data & Knowledge Engineering* **113**, 129–154 (2018)
4. Corradini, F., Morichetta, A., Polini, A., Re, B., Tiezzi, F.: Collaboration vs. choreography conformance in BPMN 2.0: From theory to practice. In: 2018 IEEE 22nd International Enterprise Distributed Object Computing Conference (EDOC). pp. 95–104. IEEE (2018)
5. Corradini, F., Muzi, C., Re, B., Rossi, L., Tiezzi, F.: BPMN 2.0 OR-Join Semantics: Global and local characterisation. *Information Systems* **105**, 101934 (2022)
6. Dechsupa, C., Vatanawood, W., Thongtak, A.: Transformation of the BPMN design model into a colored Petri net using the partitioning approach. *IEEE Access* **6**, 38421–38436 (2018)
7. Deniérou, P.M., Yoshida, N.: Multiparty session types meet communicating automata. In: European Symposium on Programming. pp. 194–213. Springer (2012)
8. van Der Aalst, W.M., Ter Hofstede, A.H., Kiepuszewski, B., Barros, A.P.: Workflow patterns. *Distributed and parallel databases* **14**(1), 5–51 (2003)
9. Dijkman, R.M., Dumas, M., Ouyang, C.: Semantics and analysis of business process models in BPMN. *Information and Software technology* **50**(12), 1281–1294 (2008)
10. Fahland, D., Völzer, H.: Dynamic skipping and blocking and dead path elimination for cyclic workflows. In: International Conference on Business Process Management. pp. 234–251. Springer (2016)
11. Houhou, S., Baarir, S., Poizat, P., Quéinnec, P., Kahloul, L.: A first-order logic verification framework for communication-parametric and time-aware BPMN collaborations. *Information systems* **104**, 101765 (2022)

12. Kräuter, T., Rutle, A., König, H., Lamo, Y.: Formalization and analysis of BPMN using graph transformation systems. In: International Conference on Graph Transformation. pp. 204–222. Springer (2023)
13. Leymann, F., Altenhuber, W.: Managing business processes as an information resource. *IBM systems journal* **33**(2), 326–348 (1994)
14. Mendling, J., Reijers, H.A., van der Aalst, W.M.: Seven process modeling guidelines (7PMG). *Information and software technology* **52**(2), 127–136 (2010)
15. Najem, T., Perucci, A.: Mapping BPMN2 service choreographies to colored petri nets. In: International Conference on Software Engineering and Formal Methods. pp. 85–100. Springer (2019)
16. Napolitano, A.: An Automatic, User-friendly Framework for Translating BPMN Diagrams to Petri Nets, <https://etd.adm.unipi.it/t/etd-03072025-112145/>
17. Object Management Group: Business Process Model and Notation (BPMN), Version 2.0. <https://www.omg.org/spec/BPMN/2.0/PDF> (Dec 2010), specification Version 2.0.2
18. Pal, S., Guanciale, R., Lanese, I., Tuosto, E., Clo, M.: Pomsets for process management: A healthcare case study. In: International Colloquium on Theoretical Aspects of Computing. pp. 378–395. Springer (2025)
19. Poizat, P., Salaün, G.: Checking the realizability of BPMN 2.0 choreographies. In: Proceedings of the 27th Annual ACM Symposium on Applied Computing. pp. 1927–1934 (2012)
20. Prinz, T.M., Choi, Y., Vetterlein, A.: Progression: A lightweight BPMN engine simplifying the execution and monitoring of process models. In: International Conference on Business Process Management. pp. 291–307. Springer (2025)
21. Ramadan, M., Elmongui, H.G., Hassan, R.: BPMN formalisation using coloured petri nets. In: Proceedings of the 2nd GSTF annual international conference on software engineering & applications (SEA 2011). pp. 83–90 (2011)
22. Red Hat: jBPM v.7.48.0. <https://www.jbpm.org> (2026)
23. Van Gorp, P., Dijkman, R.: A visual token-based formalization of BPMN 2.0 based on in-place transformations. *Information and Software Technology* **55**(2), 365–394 (2013)
24. Weidlich, M., Grosskopf, A., Barros, A.: Realising dead path elimination in BPMN. In: 2009 IEEE Conference on Commerce and Enterprise Computing. pp. 345–352. IEEE (2009)

A BPMN semantics

The left-out operational rules of Corradini's semantics [5] are:

$$\begin{array}{c}
\text{start}(e.w) \xrightarrow{1:e}_L \text{start}(e.1) \quad (\text{L-Start-NC}) \\
\text{end}(e.1) \xrightarrow{d:e}_L \text{end}(e.d) \quad (\text{L-End-NC}) \\
\text{andSplit}(e.1, E) \xrightarrow{(d:e, 1:E)}_L \text{andSplit}(e.d, \text{setLive}(E)) \quad (\text{L-AndSplit-NC}) \\
\frac{\text{isLive}(E)}{\text{andJoin}(E, e.\Sigma) \xrightarrow{(d:e, 1:E)}_L \text{andJoin}(\text{setDead}(E), e.1)} \quad (\text{L-AndJoin-NC}) \\
\frac{\neg \text{isDead}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset}{\text{andSplit}(e.d, E) \xrightarrow{d:E}_L \text{andSplit}(e.d, \text{setDead}(E))} \quad (\text{D-AndSplit}) \\
\frac{\text{isDead}(E)}{\text{andJoin}(E, e.w) \xrightarrow{d:e}_L \text{andJoin}(E, e.d)} \quad (\text{D-AndJoin}) \\
\frac{\neg \text{isDead}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset}{\text{orSplit}(e.d, E) \xrightarrow{d:E}_L \text{orSplit}(e.d, \text{setDead}(E))} \quad (\text{D-OrSplit}) \\
\frac{\text{isDead}(E)}{\text{orJoin}(E, e.w) \xrightarrow{d:e}_L \text{orJoin}(E, e.d)} \quad (\text{D-OrJoin}) \\
\frac{M_1 \xrightarrow{\alpha}_L M'_1}{M_1 \mid M_2 \xrightarrow{\alpha}_L M'_1 \mid M_2 \star \alpha} \quad (\text{M-StatusUpd}_1) \\
\frac{M_2 \xrightarrow{\alpha}_L M'_2}{M_1 \mid M_2 \xrightarrow{\alpha}_L M_1 \star \alpha \mid M'_2} \quad (\text{M-StatusUpd}_2)
\end{array}$$

where the status update function $\cdot \star \alpha$ updates edge statuses in a process according to the action α , and is defined inductively over the process structure:

$$\begin{aligned}
\emptyset \star \alpha &= \emptyset & \text{andSplit}(e, E)(e.\Sigma, E) \star \alpha &= \text{andSplit}(e, E)(e.\Sigma \star \alpha, E \star \alpha) \\
(\{e.\Sigma\} \sqcup E) \star \alpha &= \{e.\Sigma \star \alpha\} \sqcup (E \star \alpha) & (M_1 \mid M_2) \star \alpha &= M_1 \star \alpha \mid M_2 \star \alpha \\
e.\Sigma \star (w : E_1, d : E_2, 1 : E_3) &= \begin{cases} e.w & \text{if } e \in E_1 \\ e.d & \text{if } e \in E_2 \\ e.1 & \text{if } e \in E_3 \\ e.\Sigma & \text{otherwise} \end{cases}
\end{aligned}$$

Corradini et al. [5] focus on the control-flow behaviour of BPMN gateways and abstract away from atomic activities. Consequently, their semantics does not include rules for tasks. Since tasks are part of the BPMN fragment considered in this work, we extend the semantics with the two rules **L-Task-NC** and **D-Task**. The first one propagates a live token from the incoming edge to the outgoing edge, while the second propagates a dead token whenever no live token can arrive.

$$\text{task}(e.1, e'.\Sigma) \xrightarrow{(d:e, 1:e')}_L \text{task}(e.d, e'.1) \text{ (L-Task-NC)}$$

$$\text{task}(e.d, e'.w) \xrightarrow{d:e'}_L \text{task}(e.d, e'.d) \text{ (D-Task)}$$

B Auxiliary Lemmata

Definition 6 (Structural inclusion). *The structural inclusion relation $\sqsubseteq \subseteq C \times C$ is defined inductively as the smallest reflexive relation such that, for the marked BPMN processes C, C', C_1, C_2 :*

- if $C = (C_1 \mid C_2)$, then $C_1 \sqsubseteq C$ and $C_2 \sqsubseteq C$;
- if $C' \sqsubseteq C_1$ and $C_1 \sqsubseteq C$, then $C' \sqsubseteq C$ (transitivity).

Remark 3. Let C be a safe reachable marked BPMN term. If $\text{xorJoin}(E, e) \sqsubseteq C$, then there exists a $\text{xorSplit}(e, E) \sqsubseteq C$ that syntactically reaches it.

Every incoming branch of a $\text{xorJoin}(E, e)$ carries at most one token at any time, since the join fires as soon as exactly one branch is live. The upstream gateway that routes the flow into the $\text{xorJoin}(E, e)$ must therefore guarantee mutual exclusion among its outgoing branches. An $\text{andSplit}(e, E)$ activates *all* outgoing branches simultaneously, so placing an $\text{andSplit}(e, E)$ upstream of a $\text{xorJoin}(E, e)$ would cause multiple incoming branches to carry tokens at once, duplicating tokens on the output edge and violating safety. An $\text{orSplit}(e, E)$ activates a non-empty subset of its outgoing branches, so it may also activate more than one branch leading into the $\text{xorJoin}(E, e)$, again violating safety. Only a $\text{xorSplit}(e, E)$ guarantees that exactly one outgoing branch is activated at a time, preserving safety. Since C is safe by assumption, the upstream gateway must be a $\text{xorSplit}(e, E)$.

Lemma 1. *Let C be a safe reachable marked BPMN term and $\llbracket C \rrbracket = (N, M)$ its Petri net encoding with $N = (P, T, F)$. Let $\text{xorJoin}(E, e) \sqsubseteq C$ be a sub-term of C , such that e has status live. Then there exists $e' \in E$ such that for each firing $(N, M[e' \mapsto 1]) \rightarrow (N, M')$, there is a firing $(N, M) \rightarrow (N, M'')$, such that for every edge $e'' \in E(C)$, $M'(e'') = M''(e'')$.*

Proof (Proof Sketch). By Remark 3, since C is safe, there exists a $\text{xorSplit}(e, E)$ that syntactically reaches the $\text{xorJoin}(E, e)$, and every edge between them lies in the scope of ϕ , i.e. $e \in \phi(C)$ and $E \subseteq \phi(C)$.

Since e is live, the $\text{xorJoin}(E, e)$ has been triggered, and therefore the corresponding $\text{xorSplit}(e, E)$ has already fired. By the encoding of $\text{xorSplit}(e, E)$ (Table 1), firing produces a single live token in the selected branch and a dead token in each of the remaining $|E| - 1$ dead propagation places. Regardless of the components between the $\text{xorSplit}(e, E)$ and the $\text{xorJoin}(E, e)$, these dead tokens propagate through their dead propagation nets and eventually reach the incoming dead propagation places of the $\text{xorJoin}(E, e)$.

However, firing the dead propagation transition of the $\text{xorJoin}(E, e)$ requires a dead token in *every* incoming dead propagation place. The branch through which e became live has no dead counterpart: its dead propagation place $\bar{e}'$, for the corresponding $e' \in E$, never receives a token, since the branch was resolved as live, not dead. The dead propagation transition of the $\text{xorJoin}(E, e)$ is therefore never enabled. Since the net contains no cycles, no token can return to re-trigger the $\text{xorSplit}(e, E)$, so $\bar{e}'$ remains permanently unmarked, and the dead tokens accumulated on the other dead propagation places remain permanently trapped. Consequently, every transition fired after e becomes live behaves identically whether or not these dead propagation places are marked, which proves the claim. $\square$

The following lemma shows that updating C_1 according to a step $C_1 \xrightarrow{\alpha}_L C'_1$ can always be reflected onto the whole composed term $(C_1 \mid C_2)$: propagating the same status update α to C_2 via the $\star$ operator does not break the correspondence between the BPMN semantics and its Petri net encoding. Concretely, we show that the marking obtained by firing the corresponding transition in the full net N still satisfies the two coherence conditions of Theorem 1 with respect to the encoding of the updated term $(C'_1 \mid C_2 \star \alpha)$. The proof proceeds by cases on how α affects C_2 ; in the case where the update reaches an $\text{xorJoin}(E, e)$ whose dead propagation net may already carry tokens left over from a previously resolved sibling branch, we rely on Lemma 1 to argue that such tokens are harmless and do not affect the enabledness of subsequent transitions, hence do not threaten coherence.

Lemma 2. *Let $C = (C_1 \mid C_2)$ be a reachable marked BPMN term, $(N, M) = \llbracket C \rrbracket$, $(N_1, M_1) = \llbracket C_1 \rrbracket$, and $(N_1, M'_1) = \llbracket C'_1 \rrbracket$. If $C_1 \xrightarrow{\alpha}_L C'_1$ and $(N_1, M_1) \rightarrow (N_1, M'_1)$ with $M'_1(e) = M_1(e)$ for all $e \in E(C_1)$ and $M'_1(\bar{e}) \geq M_1(\bar{e})$ for all $e \in \phi(C_1)$, then there exists a firing $(N, M) \rightarrow (N, M'')$ with $M''(e) = M'(e)$ for all $e \in E(C)$ and $M''(\bar{e}) \geq M'(\bar{e})$ for all $e \in \phi(C)$, where $(N, M') = \llbracket (C'_1 \mid C_2 \star \alpha) \rrbracket$.*

Proof. We proceed by structural induction on $C = (C_1 \mid C_2)$. We show the most interesting cases; the others are similar. In each case, the firing in (N, M) is the same transition that fires in (N_1, M_1) , since N_1 is a subnet of N . The place update is automatically reflected in N_2 for shared places, which corresponds to the $\star$ operator at the BPMN level.

Start: Let $C_1 = \text{start}(e.w)$ and $C_1 \xrightarrow{\alpha}_L C'_1$ via rule L-Start-NC, with $\alpha = 1 : e$.

By definition, $M_1(p_s) = 1$ and $M_1(e) = 0$. In (N, M) , the same holds since $N_1 \subseteq N$: transition t_s fires, consuming the token from p_s and producing one in e , yielding M'' with $M''(p_s) = 0$ and $M''(e) = 1$. Since $C_2 \star \alpha = C_2 \star 1 : e = C'_2$, by definition of $\star$, $M'(e) = 1$ in $\llbracket (C'_1 \mid C'_2) \rrbracket$. The first condition holds: $M''(e) = 1 = M'(e)$ for all $e \in E(C)$. The second condition holds because $\text{start}(e)$ introduces no places in the dead propagation net, hence $M''(\bar{e}) = 0 = M'(\bar{e})$.

End: Let $C_1 = \text{end}(e.1)$ and $C_1 \xrightarrow{\alpha}_L C'_1$ via rule L-End-NC, with $\alpha = \mathbf{d} : e$. By definition, $M_1(e) = 1$. Transition t_e fires in (N, M) , consuming the token from e , yielding M'' with $M''(e) = 0$. Since $C_2 \star \alpha = C_2 \star \mathbf{d} : e = C'_2$, by definition of $\star$, $M'(e) = 0$ in $\llbracket (C'_1 \mid C'_2) \rrbracket$. The first condition holds: $M''(e) = 0 = M'(e)$ for all $e \in E(C)$. The second condition holds trivially: e is live in C_1 , so by safety $M(\bar{e}) = 0$; firing t_e produces no token in the dead propagation net, hence $M''(\bar{e}) = 0 = M'(\bar{e})$.

Task: Let $C_1 = \text{task}(e.1, e')$ and $C_1 \xrightarrow{\alpha}_L C'_1$ via rule L-Task-NC, with $\alpha = \mathbf{d} : e, 1 : e'$. By definition, $M_1(e) = 1$ and $M_1(e') = 0$. Transition t fires in (N, M) , consuming the token from e and producing one in e' , yielding M'' with $M''(e) = 0$ and $M''(e') = 1$. Since $C_2 \star \alpha = C_2 \star (\mathbf{d} : e, 1 : e') = C'_2$, by definition of $\star$, $M'(e) = 0$ and $M'(e') = 1$ in $\llbracket (C'_1 \mid C'_2) \rrbracket$. The first condition holds: $M''(e) = 0 = M'(e)$ and $M''(e') = 1 = M'(e')$. The second condition holds trivially: e is live in C_1 and e' is live in C'_1 , so by safety $M(\bar{e}) = 0$ and $M'(\bar{e}') = 0$; firing t produces no token in the dead propagation net, hence $M''(\bar{e}) = 0 = M'(\bar{e})$ and $M''(\bar{e}') = 0 = M'(\bar{e}')$. The case $\Sigma = \mathbf{d}$ uses rule D-Task, and is analogous.

OR-split: Let $C_1 = \text{orSplit}(e.1, E)$ and $C_1 \xrightarrow{\alpha}_L C'_1$ via rule L-OrSplit-NC, with $E_1 \neq \emptyset$, $E_2 = E \setminus E_1$, and $\alpha = \mathbf{d} : \{e\} \cup E_2, 1 : E_1$. By definition, $M_1(e) = 1$ and $M_1(e') = 0$ for all $e' \in E$. Transition t_{E_1} fires in (N, M) , consuming the token from e and producing one token in each $e' \in E_1$ and one token in each $\bar{e}'' \in E_2$, yielding M'' with $M''(e) = 0$, $M''(e') = 1$ for all $e' \in E_1$, and $M''(\bar{e}'') = 1$ for all $e'' \in E_2$. Since $C_2 \star \alpha = C'_2$, by definition of $\star$, M' agrees with M'' on all shared edges. The first condition holds: $M''(e''') = M'(e''')$ for all $e''' \in E(C)$. For the second condition, each $e'' \in E_2$ may or may not be a frontier dotted edge, so $M'(\bar{e}'') \in \{0, 1\}$; since $M''(\bar{e}'') = 1$, we have $M''(\bar{e}'') \geq M'(\bar{e}'')$. For $e' \in E_1$, by safety $M'(\bar{e}') = 0$ and $M''(\bar{e}') = 0$, so equality holds trivially. Together, both conditions are satisfied, concluding the case.

OR-join: Let $C_1 = \text{orJoin}(E, e)$ and $C_1 \xrightarrow{\alpha}_L C'_1$ via rule L-OrJoin-NC, with $E_1 \neq \emptyset$, $E_2 = E \setminus E_1$, and $\alpha = 1 : e$. By definition, $M_1(e') = 1$ for all $e' \in E_1$, $M_1(e'') = 1$ for all $e'' \in E_2$, and $M_1(e) = 0$. Transition t_{E_1} fires in (N, M) , consuming all tokens from E_1 and $\bar{E}_2$ and producing one token in e , yielding M'' with $M''(e) = 1$ and $M''(e') = 0$, $M''(\bar{e}'') = 0$ for all $e' \in E_1$, $e'' \in E_2$. Since $C_2 \star \alpha = C'_2$, by definition of $\star$, $M'(e) = 1$ in $\llbracket (C'_1 \mid C'_2) \rrbracket$. The first condition holds: $M''(e''') = M'(e''')$ for all $e''' \in E(C)$. For the second condition, e may or may not be a frontier dotted edge, so $M'(\bar{e}) \in \{0, 1\}$; since $M''(\bar{e}) = 0$, we need $M''(\bar{e}) \geq M'(\bar{e})$, which holds since e is live in C'_1 and by safety $M'(\bar{e}) = 0$. Together, both conditions are satisfied, concluding the case.

M-StatusUpd₁: Let $C_1 = (C_1^A \mid C_1^B)$, so $C = ((C_1^A \mid C_1^B) \mid C_2)$. Assume $C_1^A \xrightarrow{\alpha}_L C_1^{A'}$ via rule M-StatusUpd₁, with action α ; the other case is symmetric. Then $C \xrightarrow{\alpha}_L (C_1^{A'} \mid (C_1^B \star \alpha \mid C_2 \star \alpha)) = (C_1^{A'} \mid (C_1^B \mid C_2) \star \alpha)$. By inductive hypothesis applied to C_1^A and $(C_1^B \mid C_2)$, the firing $(N_1^A, M_1^A) \rightarrow (N_1^{A'}, M_1^{A'})$ satisfies the two conditions with respect to $\llbracket C_1^{A'} \rrbracket$. If this update

reaches an $\text{xorJoin}(E, e)$ whose dead propagation net already carries tokens left over from a previously resolved sibling branch, Lemma 1 ensures such tokens are harmless, since they do not affect the enabledness of any subsequent transition, and hence do not threaten coherence. By Lemma 2 there exists a firing $(N, M) \rightarrow (N, M'')$ such that $M''(e) = M'(e)$ for all $e \in E(C)$ and $M''(\bar{e}) \geq M'(\bar{e})$ for all $e \in \phi(C)$, where $(N, M') = \llbracket (C_1^{A'} \mid (C_1^B \mid C_2) \star \alpha) \rrbracket$, concluding the case. $\square$

C Correctness

Theorem 1 (Correctness). *Let C be a reachable marked BPMN term, and $\llbracket C \rrbracket = (N, M)$ be its Petri net encoding and marking. If $C \xrightarrow{\alpha}_L C'$ then there is a firing $(N, M) \rightarrow (N, M')$ such that, assuming $\llbracket C' \rrbracket = (N, M'')$, $M'(e) = M''(e)$ for all $e \in E(C)$ and $M''(\bar{e}) \leq M'(\bar{e})$ for all $e \in \phi(C)$.*

Proof. Recall that we only consider safe and reachable marked BPMN terms. Safety is exploited to ensure that in case of live token propagation, outgoing edges initially have wait status. By induction on the height of the derivation of $C \xrightarrow{\alpha}_L C'$, with a case analysis on the applied rules.

L-Start-NC: Let $\text{PN}(\text{start}(e)) = (P, T, F)$ with $P = \{p_s, e\}$, $T = \{t_s\}$, and $F = \{(p_s, t_s), (t_s, e)\}$. Recall the L-Start-NC rule:

$$\frac{}{\text{start}(e.w) \xrightarrow{\alpha}_L \text{start}(e.1)}$$

where $C = \text{start}(e.w)$, $C' = \text{start}(e.1)$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. By definition, $M(p_s) = 1$ and $M(e) = 0$, while $M'(p_s) = 0$ and $M'(e) = 1$. The transition t_s has $\bullet t_s = \{p_s\}$ and $t_s \bullet = \{e\}$; therefore it is enabled under M . Firing t_s produces one token in e , yielding a marking M'' such that $M''(p_s) = 0$ and $M''(e) = 1$. The first condition holds since $M''(e) = M'(e)$. The second condition holds vacuously: $\text{start}(e)$ introduces no dead propagation places, since its output edge cannot be a dotted edge by definition of ϕ .

L-End-NC: Let $\text{PN}(\text{end}(e)) = (P, T, F)$ with $P = \{e\}$, $T = \{t_e\}$, and $F = \{(e, t_e)\}$. Recall the L-End-NC rule:

$$\frac{}{\text{end}(e.1) \xrightarrow{\alpha}_L \text{end}(e.d)}$$

where $C = \text{end}(e.1)$, $C' = \text{end}(e.d)$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. By definition, $M(e) = 1$ and $M'(e) = 0$. The transition t_e has $\bullet t_e = \{e\}$ and $t_e \bullet = \emptyset$; therefore it is enabled under M . Firing t_e removes one token from e , yielding a marking M'' such that $M''(e) = 0$. The first condition holds since $M''(e) = M'(e)$. The second condition holds trivially: e is live in C , so by safety $M(\bar{e}) = 0$; firing t_e produces no token on the dead propagation net, hence $M''(\bar{e}) = 0 = M'(\bar{e})$.

L-Task-NC: Let $\text{PN}(\text{task}(e, e')) = (P, T, F)$ with $P = \{e, e'\}$, $T = \{t\}$, and $F = \{(e, t), (t, e')\}$. Recall the L-Task-NC rule:

$$\frac{}{\text{task}(e.1, e'.\Sigma) \xrightarrow{\alpha}_L \text{task}(e.d, e'.1)}$$

where $C = \text{task}(e.1, e'.\Sigma)$, $C' = \text{task}(e.d, e'.1)$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. For safety, $e'.\Sigma$ must be wait. By definition, $M(e) = 1$, $M(e') = 0$, while $M'(e) = 0$ and $M'(e') = 1$. The transition t has $\bullet t = \{e\}$ and $t \bullet = \{e'\}$; hence it is enabled under M . Firing t removes one token from e and adds one token in e' , yielding a marking M'' such that $M''(e) = 0$ and $M''(e') = 1$. The first condition holds since $M''(e) = M'(e)$ and $M''(e') = M'(e')$. The second condition holds trivially: e is live in C , so by safety $M(\bar{e}) = 0$; e' is live in C' , so by safety $M'(\bar{e}') = 0$; firing t produces no token on the dead propagation net, hence $M''(\bar{e}) = 0 = M'(\bar{e})$ and $M''(\bar{e}') = 0 = M'(\bar{e}')$.

L-AndSplit-NC: Let $\text{PN}(\text{andSplit}(e, E)) = (P, T, F)$ with $P = \{e\} \cup E$, $T = \{t_a\}$, and $F = \{(e, t_a)\} \cup \{(t_a, e') \mid e' \in E\}$. Recall the L-AndSplit-NC rule:

$$\frac{}{\text{andSplit}(e.1, E) \xrightarrow{\alpha}_L \text{andSplit}(e.d, \text{setLive}(E))}$$

where we let $C = \text{andSplit}(e.1, E)$, $C' = \text{andSplit}(e.d, \text{setLive}(E))$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. For safety, for every $e.\Sigma$ in E , the statuses must be wait. By definition, $M(e) = 1$ and $M(e') = 0$ for all $e' \in E$, while $M'(e) = 0$ and $M'(e') = 1$ for all $e' \in E$. The transition t_a has $\bullet t_a = \{e\}$ and $t_a \bullet = E$; hence it is enabled under M . Firing t_a consumes the token from e and produces one token in each $e' \in E$, yielding a marking M'' such that $M''(e) = 0$ and $M''(e') = 1$ for all $e' \in E$. The first condition holds since $M''(e) = M'(e)$ and $M''(e') = M'(e')$ for all $e' \in E$. The second condition holds trivially: e is live in C , so by safety $M(\bar{e}) = 0$; every $e' \in E$ is live in C' , so by safety $M'(\bar{e}') = 0$; firing t_a produces no token on the dead propagation net, hence $M''(\bar{e}) = 0 = M'(\bar{e})$ and $M''(\bar{e}') = 0 = M'(\bar{e}')$ for all $e' \in E$.

L-AndJoin-NC: Let $\text{PN}(\text{andJoin}(E, e)) = (P, T, F)$ with $P = E \cup \{e\}$, $T = \{t_a\}$, and $F = \{(e', t_a) \mid e' \in E\} \cup \{(t_a, e)\}$. Recall the L-AndJoin-NC rule:

$$\frac{\text{isLive}(E)}{\text{andJoin}(E, e.\Sigma) \xrightarrow{\alpha}_L \text{andJoin}(\text{setDead}(E), e.1)}$$

where we let $C = \text{andJoin}(E, e.\Sigma)$, $C' = \text{andJoin}(\text{setDead}(E), e.1)$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. For safety, $e.\Sigma$ must be wait. By definition, $M(e') = 1$ for all $e' \in E$ and $M(e) = 0$, while $M'(e') = 0$ for all $e' \in E$ and $M'(e) = 1$. The transition t_a has $\bullet t_a = E$ and $t_a \bullet = \{e\}$; since every place in E holds a token under M , t_a is enabled. Firing t_a consumes one token from each $e' \in E$ and produces one token in e , yielding a marking M'' such that $M''(e') = 0$ for all $e' \in E$ and $M''(e) = 1$. The first condition holds since $M''(e') = M'(e')$ for all $e' \in E$ and $M''(e) = M'(e)$. The second condition

holds trivially: every $e' \in E$ is live in C , so by safety $M(\bar{e}') = 0$; e is live in C' , so by safety $M'(\bar{e}) = 0$; firing t_a produces no token on the dead propagation net, hence $M''(\bar{e}') = 0 = M'(\bar{e}')$ for all $e' \in E$ and $M''(\bar{e}) = 0 = M'(\bar{e})$.

L-XorSplit-NC: Let $\text{PN}(\text{xorSplit}(e, E)) = (P, T, F)$ and recall that the encoding introduces, for every $e' \in E$, a transition $t_{e'}$ with $\bullet t_{e'} = \{e\}$ and $t_{e'} \bullet = \{e'\} \cup \{\bar{e}'' \mid e'' \in E, e'' \neq e'\}$. Recall also the L-XorSplit-NC rule:

$$\text{xorSplit}(e.1, \{e_s.\Sigma\} \sqcup E') \xrightarrow{\alpha}_L \text{xorSplit}(e.d, \{e_s.1\} \sqcup \text{setDead}(E'))$$

where $C = \text{xorSplit}(e.1, \{e_s.\Sigma\} \sqcup E')$, $C' = \text{xorSplit}(e.d, \{e_s.1\} \sqcup \text{setDead}(E'))$, with $E = \{e_s\} \sqcup E'$ the full set of outgoing edges, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. For safety, $e_s.\Sigma$ must be wait. By definition, $M(e) = 1$ and $M(e'') = 0$ for all $e'' \in E$, while $M'(e) = 0$, $M'(e_s) = 1$, $M'(e'') = 0$ and $M'(\bar{e}'') = 1$ for all $e'' \in E'$. Since $\bullet t_{e''} = \{e\}$ for all $e'' \in E$, all transitions are enabled under M ; following the rule above we choose to fire transition t_{e_s} , which consumes the token from e and produces, in a single step, one token in e_s and one token in $\bar{e}''$ for every $e'' \in E'$. The resulting marking M'' satisfies: $M''(e) = 0$, $M''(e_s) = 1$, $M''(e'') = 0$ and $M''(\bar{e}'') = 1$ for all $e'' \in E'$. The first condition holds since $M''(e''') = M'(e''')$ for all $e''' \in E(C)$. The second condition holds trivially: e_s is live in C' , so by safety $M'(\bar{e}_s) = 0$; firing t_{e_s} produces no token on $\bar{e}_s$, hence $M''(\bar{e}_s) = 0 = M'(\bar{e}_s)$. For all $e'' \in E'$, the Petri net produces $M''(\bar{e}'') = 1$, while $M'(\bar{e}'') \in \{0, 1\}$ depending on whether e'' is a frontier dotted edge; hence $M''(\bar{e}'') \geq M'(\bar{e}'')$. Together, both conditions of the theorem are satisfied, concluding the case.

L-XorJoin-NC: Let $\text{PN}(\text{xorJoin}(E, e)) = (P, T, F)$ and recall that the encoding introduces, for every $e'' \in E$, a transition $t_{e''}$ with $\bullet t_{e''} = \{e''\}$ and $t_{e''} \bullet = \{e\}$. Recall also the L-XorJoin-NC rule:

$$\text{xorJoin}(\{e_f.1\} \sqcup E', e.\Sigma) \xrightarrow{\alpha}_L \text{xorJoin}(\{e_f.d\} \sqcup \text{setDead}(E'), e.1)$$

where $C = \text{xorJoin}(\{e_f.1\} \sqcup E', e.\Sigma)$, $C' = \text{xorJoin}(\{e_f.d\} \sqcup \text{setDead}(E'), e.1)$, with $E = \{e_f\} \sqcup E'$ the full set of incoming edges, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. For safety, $e.\Sigma$ must be wait. By definition of the encoding, $M(e_f) = 1$, $M(e'') = 0$ for all $e'' \in E'$, and $M(e) = 0$, while $M'(e_f) = 0$, $M'(e'') = 0$ for all $e'' \in E'$, and $M'(e) = 1$. Since $\bullet t_{e''} = \{e''\}$ for all $e'' \in E$, exactly t_{e_f} is enabled under M ; following the rule above we fire transition t_{e_f} , which consumes the token from e_f and produces one token in e . The resulting marking M'' satisfies: $M''(e_f) = 0$, $M''(e'') = 0$ for all $e'' \in E'$, and $M''(e) = 1$. The first condition holds since $M''(e''') = M'(e''')$ for all $e''' \in E(C)$. The second condition holds trivially: e is live in C' , so by safety $M'(\bar{e}) = 0$; the XOR-join encoding introduces no token on the dead propagation net in its postset, hence $M''(\bar{e}) = M(\bar{e}) = 0 = M'(\bar{e})$. For all $e'' \in E'$, firing t_{e_f} produces no token on the dead propagation net, hence $M''(\bar{e}'') = M(\bar{e}'')$; since markings are non-negative and $M'(\bar{e}'') = 0$, we get $M''(\bar{e}'') \geq M'(\bar{e}'')$. Together, both conditions of the theorem are satisfied, concluding the case.

L-OrSplit-NC: Let $\text{PN}(\text{orSplit}(e, E))$ and recall that the encoding introduces, for every $S \in \mathcal{P}_\emptyset(E)$, a transition t_S with $\bullet t_S = \{e\}$ and $t_S \bullet = \{e' \mid e' \in S\} \cup \{\bar{e}'' \mid e'' \in E \setminus S\}$. Recall also the L-OrSplit-NC rule:

$$\frac{E_1 \neq \emptyset \quad E = \text{setLive}(E_1) \sqcup \text{setDead}(E_2)}{\text{orSplit}(e.1, E_1 \sqcup E_2) \xrightarrow{(\text{d:}\{e\} \sqcup E_2, 1:E_1)}_L \text{orSplit}(e.d, E)}$$

where $C = \text{orSplit}(e.1, E_1 \sqcup E_2)$, $C' = \text{orSplit}(e.d, E)$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. The statuses of every edge $e'.\Sigma \in E$ are wait. Otherwise, this will go against the safety assumption. Then, by definition, the marking of $\llbracket C \rrbracket$ is $M(e) = 1$ and for all $e' \in E$, the corresponding places e' , $\bar{e}'$ are empty. Since $M(e) = 1$, all transitions t_S , for all $S \in \mathcal{P}_\emptyset(E)$, are enabled; following the rule above we choose to fire transition t_{E_1} , which consumes the token from e and produces, in a single step, one token in e' for every $e' \in E_1$, and one token in $\bar{e}''$ for every $e'' \in E_2$. The resulting marking M'' satisfies: $M''(e) = 0$, $M''(e') = 1$ and $M''(\bar{e}') = 0$ for all $e' \in E_1$, $M''(e'') = 0$ and $M''(\bar{e}'') = 1$ for all $e'' \in E_2$. It remains to verify the two conditions of the theorem. For the first condition, the live places match by construction: $M''(e''') = M'(e''')$ for all $e''' \in E(C)$. For the second condition, the outgoing edges in E_2 may or may not be frontier dead edges in $\llbracket C' \rrbracket$; hence $\llbracket C' \rrbracket$ may carry no token on $\bar{e}''$ for some $e'' \in E_2$. For every $e'' \in E_2$, the Petri net produces $M''(\bar{e}'') = 1$, while $\llbracket C' \rrbracket$ carries at most one token on $\bar{e}''$, so $M''(\bar{e}'') \geq M'(\bar{e}'')$. For every $e' \in E_1$, the Petri net produces $M''(e') = 0$; by safety, e' being live in C' implies $M'(e') = 0$, hence $M''(e') \geq M'(e')$ holds trivially. Together, both conditions of the theorem are satisfied, concluding the case.

L-OrJoin-NC: Let $\text{PN}(\text{orJoin}(E, e))$ and recall that the encoding introduces, for every $S \in \mathcal{P}_\emptyset(E)$, a transition t_S with $\bullet t_S = \{e' \mid e' \in S\} \cup \{\bar{e}'' \mid e'' \in E \setminus S\}$ and $t_S \bullet = \{e\}$. Recall also the L-OrJoin-NC rule:

$$\frac{\neg \text{isWait}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} \neq \emptyset}{\text{orJoin}(E, e.\Sigma) \rightarrow_L \text{orJoin}(\text{setDead}(E), e.1)}$$

where $C = \text{orJoin}(E, e.\Sigma)$, $C' = \text{orJoin}(\text{setDead}(E), e.1)$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. The status of edge $e.\Sigma$ is wait. Otherwise, this will go against the safety assumption. Since $\neg \text{isWait}(E)$, every branch $e'' \in E$ is either live or dead, i.e. exactly one of e'' or $\bar{e}''$ holds a token in the marking M . Let $E_1 \subseteq E$ be the set of live edges and $E_2 = E \setminus E_1$ the set of dead edges, with $E_1 \neq \emptyset$ by hypothesis. Then, by definition, the marking of $\llbracket C \rrbracket$ is $M(e') = 1$ for all $e' \in E_1$, $M(\bar{e}'') = 1$ for all $e'' \in E_2$, and all other places are empty. Since the preset of t_{E_1} is exactly $\{e' \mid e' \in E_1\} \cup \{\bar{e}'' \mid e'' \in E_2\}$, the transition t_{E_1} is enabled under M ; following the rule above we choose to fire transition t_{E_1} , which consumes, in a single step, the token from each $e' \in E_1$ and from each $\bar{e}'' \in E_2$, and produces one token in e . The resulting marking M'' satisfies: $M''(e) = 1$, $M''(e'') = 0$ and $M''(\bar{e}'') = 0$ for all $e'' \in E$. It remains to verify the two conditions of the theorem. For the

first condition, the live places match by construction: $M''(e''') = M'(e''')$ for all $e''' \in E(C)$. For the second condition, the output edge e may or may not be a frontier dead edge in $\llbracket C' \rrbracket$; hence $\llbracket C' \rrbracket$ may carry no token on $\bar{e}$. The Petri net produces $M''(\bar{e}) = 0$, while $\llbracket C' \rrbracket$ carries at most one token on $\bar{e}$, so $M''(\bar{e}) \geq M'(\bar{e})$ holds trivially. For all $e'' \in E$, the Petri net produces $M''(\bar{e}'') = 0$; by safety, e being live in C' implies $M'(\bar{e}'') = 0$, hence $M''(\bar{e}'') \geq M'(\bar{e}'')$ holds trivially. Together, both conditions of the theorem are satisfied, concluding the case.

D-Task: Let $\text{PN}(\text{task}(e, e')) = (P, T, F)$ where $P = \{\bar{e}, \bar{e}'\}$, $T = \{\bar{t}\}$, and $F = \{(\bar{e}, \bar{t}), (\bar{t}, \bar{e}')\}$. Recall the D-Task rule:

$$\frac{}{\text{task}(e.d, e'.w) \xrightarrow{\alpha}_L \text{task}(e.d, e'.d)}$$

where $C = \text{task}(e.d, e'.\Sigma)$, $C' = \text{task}(e.d, e'.d)$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. By definition, $M(\bar{e}) = 1$ and $M(\bar{e}') = 0$, while $M'(\bar{e}) = 0$ and $M'(\bar{e}') \in \{0, 1\}$ depending on whether e' is a frontier dotted edge. The transition $\bar{t}$ has $\bullet\bar{t} = \{\bar{e}\}$; hence it is enabled under M ; following the rule above we fire transition $\bar{t}$, which consumes one token from $\bar{e}$ and produces one token in $\bar{e}'$. The resulting marking M'' satisfies: $M''(\bar{e}) = 0$ and $M''(\bar{e}') = 1$. It remains to verify the two conditions of the theorem. For the first condition, no live places are involved in this rule, so $M''(e''') = M'(e''') = 0$ for all $e''' \in E(C)$. For the second condition, the Petri net produces $M''(\bar{e}') = 1$, while $M'(\bar{e}') \in \{0, 1\}$, hence $M''(\bar{e}') \geq M'(\bar{e}')$. Together, both conditions of the theorem are satisfied, concluding the case.

D-AndSplit, D-XorSplit, D-OrSplit: Let $\text{PN}(\text{Split}(e, E)) = (P, T, F)$, where $P = \{\bar{e}\} \cup \{\bar{e}' \mid e' \in E\}$, $T = \{\bar{t}_a\}$, and $F = \{(\bar{e}, \bar{t}_a)\} \cup \{(\bar{t}_a, \bar{e}') \mid e' \in E\}$. Recall the dead-propagation rule:

$$\frac{\neg \text{isDead}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset}{\text{Split}(e.d, E) \xrightarrow{\alpha}_L \text{Split}(e.d, \text{setDead}(E))}$$

where Split can be an AND-split, XOR-split, and OR-split $C = \text{Split}(e.d, E)$, $C' = \text{Split}(e.d, \text{setDead}(E))$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. Since the three rules are identical, we give a single proof. By definition of the dead-propagation net, $M(\bar{e}) = 1$ and $M(\bar{e}') = 0$ for all $e' \in E$, while $M'(\bar{e}') \in \{0, 1\}$ depending on whether e' is a frontier dotted edge. The transition $\bar{t}_a$ has $\bullet\bar{t}_a = \{\bar{e}\}$; hence it is enabled under M ; following the rule above we fire transition $\bar{t}_a$, which consumes one token from $\bar{e}$ and produces one token in every $\bar{e}'$, for $e' \in E$. The resulting marking M'' satisfies $M''(\bar{e}) = 0$ and $M''(\bar{e}') = 1$ for all $e' \in E$. It remains to verify the two conditions of the theorem. For the first condition, no live places are involved in this rule. For the second condition, the Petri net produces $M''(\bar{e}') = 1$, while $M'(\bar{e}') \in \{0, 1\}$, hence $M''(\bar{e}') \geq M'(\bar{e}')$ for all $e' \in E$. Together, both conditions of the theorem are satisfied, concluding the case.

D-AndJoin, D-XorJoin, D-OrJoin: Let $\text{PN}(\text{Join}(E, e)) = (P, T, F)$, where $P = \{\bar{e}' \mid e' \in E\} \cup \{\bar{e}\}$, $T = \{\bar{t}_a\}$, and $F = \{(\bar{e}', \bar{t}_a) \mid e' \in E\} \cup \{(\bar{t}_a, \bar{e})\}$. Recall

the dead-propagation rule:

$$\frac{\text{isDead}(E)}{\text{Join}(E, e.w) \xrightarrow{\alpha}_L \text{Join}(E, e.d)}$$

where Join can be an AND-join, XOR-join, and OR-join $C = \text{Join}(E, e.w)$, $C' = \text{Join}(E, e.d)$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. Since the three rules are identical, we give a single proof. By definition of the dead-propagation net, $M(e') = 1$ for all $e' \in E$, while $M(\bar{e}) = 0$. Moreover, $M'(\bar{e}) \in \{0, 1\}$ depending on whether e is a frontier dotted edge. The transition $\bar{t}_a$ has $\bullet\bar{t}_a = \{e' \mid e' \in E\}$; hence it is enabled under M ; following the rule above we fire transition $\bar{t}_a$, which consumes one token from each e' and produces one token in $\bar{e}$. The resulting marking M'' satisfies $M''(e') = 0$ for all $e' \in E$ and $M''(\bar{e}) = 1$. It remains to verify the two conditions of the theorem. For the first condition, no live places are involved in this rule. For the second condition, the Petri net produces $M''(\bar{e}) = 1$, while $M'(\bar{e}) \in \{0, 1\}$, hence $M''(\bar{e}) \geq M'(\bar{e})$. Together, both conditions of the theorem are satisfied, concluding the case.

M-StatusUpd₁: The applied rule is

$$\frac{C_1 \xrightarrow{\alpha}_L C'_1}{(C_1 \mid C_2) \xrightarrow{\alpha}_L (C'_1 \mid C_2 \star \alpha)}$$

where $C = (C_1 \mid C_2)$, $C' = (C'_1 \mid C_2 \star \alpha)$, $\llbracket C \rrbracket = (N, M)$, and $\llbracket C' \rrbracket = (N, M')$. Since C is reachable, so is its sub-term C_1 ; moreover, the derivation of $C_1 \xrightarrow{\alpha}_L C'_1$ has strictly smaller height than that of $C \xrightarrow{\alpha}_L C'$, so the inductive hypothesis applies to it. Let $\llbracket C_1 \rrbracket = (N_1, M_1)$ and $\llbracket C'_1 \rrbracket = (N_1, M'_1)$. By inductive hypothesis, there exists a firing $(N_1, M_1) \rightarrow (N_1, M''_1)$ such that $M''_1(e) = M'_1(e)$ for all $e \in E(C_1)$ and $M''_1(\bar{e}) \geq M'_1(\bar{e})$ for all $e \in \phi(C_1)$. By Lemma 2, applied to this firing, there exists a firing $(N, M) \rightarrow (N, M'')$ such that $M''(e) = M'(e)$ for all $e \in E(C)$ and $M''(\bar{e}) \geq M'(\bar{e})$ for all $e \in \phi(C)$, which are exactly the two conditions required by the theorem, concluding the case. The case **M-StatusUpd₂** is symmetric, obtained by exchanging the roles of C_1 and C_2 throughout (and applying the symmetric counterpart of Lemma 2). $\square$

D Completeness

Theorem 2 (Completeness). *Let C be a reachable marked BPMN term, and $\llbracket C \rrbracket = (N, M)$ its Petri net encoding and marking. If $(N, M) \rightarrow (N, M'')$, then there exists a reachable marked BPMN term C' such that $C \xrightarrow{\alpha}_L C'$ and $(N, M') = \llbracket C' \rrbracket$, for all $e \in E(C)$, $M'(e) = M''(e)$ and for all $e \in \phi(C)$, $M''(\bar{e}) \geq M'(\bar{e})$.*

Proof. Recall that we only consider safe and reachable marked BPMN terms. Safety is exploited to ensure that in case of live token propagation, outgoing edges

initially have wait status. We consider here first the cases in which incoming edges are live; the case in which edges are dead, triggering the dead propagation transition, is discussed at the bottom. By structural induction on the BPMN terms C :

Start: Let $\text{PN}(\text{start}(e)) = (P, T, F)$ with $P = \{p_s, e\}$, $T = \{t\}$, and $F = \{(p_s, t), (t, e)\}$. Recall the **L-Start-NC** rule: $\text{start}(e.w) \xrightarrow{\alpha_L} \text{start}(e.1)$ where $C = \text{start}(e.w)$, $C' = \text{start}(e.1)$, $\text{PN}(C) = (N, M)$, and $\text{PN}(C') = (N, M')$. The only transition is t , so the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of t . By definition, $M(p_s) = 1$ and $M(e) = 0$, so t is enabled. Firing t yields M'' with $M''(p_s) = 0$ and $M''(e) = 1$. By rule **L-Start-NC** we have $C \xrightarrow{\alpha_L} C'$, and by definition $M'(e) = 1 = M''(e)$, so the first condition holds. The second condition holds because the **start**(e) introduces no complementary places by definition of ϕ , concluding the case.

End: Let $\text{PN}(\text{end}(e.\Sigma)) = (P, T, F)$ with $P = \{e\}$, $T = \{t_e\}$, and $F = \{(e, t_e)\}$, so $\bullet t_e = \{e\}$ and $t_e \bullet = \emptyset$. The only transition in the net is t_e , so the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of t_e . Since $\bullet t_e = \{e\}$, the transition t_e is enabled under M iff $M(e) \geq 1$. The marking encoding gives $M(e) = 1$ with $\Sigma = 1$, which is the only reachable case for an end event before it has fired. Firing t_e yields M'' with $M''(e) = M(e) - 1 = 0$. Setting $C' = \text{end}(e.d)$, by rule **L-End-NC** we have $C \xrightarrow{\alpha_L} C'$, and $\text{PN}(C') = (N, M')$. By definition, $M'(e) = 0 = M''(e)$, so the first condition holds. The second condition holds because the end event's transition t_e has empty postset, so no dead propagation place is affected by this firing, concluding the case.

Task: Let $\text{PN}(\text{task}(e.\Sigma, e'.\Sigma')) = (P, T, F)$ with $P = \{e, e'\}$, $T = \{t_t\}$, and $F = \{(e, t_t), (t_t, e')\}$, so $\bullet t_t = \{e\}$ and $t_t \bullet = \{e'\}$. Since $\Sigma = 1$, the transition t_t is the only one enabled under M , so the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of t_t . The marking encoding gives $M(e) = 1$, which is the only reachable case for a task before it has fired. Firing t_t yields M'' with $M''(e) = M(e) - 1 = 0$ and $M''(e') = M(e') + 1 = 1$. Setting $C' = \text{task}(e.d, e'.1)$, by rule **L-Task-NC** we have $C \xrightarrow{\alpha_L} C'$, and $\text{PN}(C') = (N, M')$. By definition, $M'(e) = 0 = M''(e)$ and $M'(e') = 1 = M''(e')$, so the first condition holds. The second condition holds because the firing t_t affects no dead propagation place, concluding the case.

AndSplit: Let $\text{PN}(\text{andSplit}(e.\Sigma, E)) = (P, T, F)$ with $P = \{e\} \cup E$, $T = \{t_a\}$, and $F = \{(e, t_a)\} \cup \{(t_a, e') \mid e' \in E\}$, so $\bullet t_a = \{e\}$ and $t_a \bullet = E$. Since $\Sigma = 1$, the transition t_a is the only one enabled under M , so the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of t_a . The marking encoding gives $M(e) = 1$, which is the only reachable case for an and-split before it has fired. Firing t_a yields M'' with $M''(e) = M(e) - 1 = 0$ and $M''(e') = M(e') + 1 = 1$ for all $e' \in E$. Setting $C' = \text{andSplit}(e.d, \text{setLive}(E))$, by rule **L-AndSplit-NC** we have $C \xrightarrow{\alpha_L} C'$, and $\text{PN}(C') = (N, M')$. By definition, $M'(e) = 0 = M''(e)$ and $M'(e') = 1 = M''(e')$ for all $e' \in E$, so the first condition holds. The second condition holds vacuously: firing t_a affects no dead propagation place, concluding the case.

AndJoin: Let $\text{PN}(\text{andJoin}(E, e, \Sigma)) = (P, T, F)$ with $P = E \cup \{e\}$, $T = \{t_a\}$, and $F = \{(e'', t_a) \mid e'' \in E\} \cup \{(t_a, e)\}$, so $\bullet t_a = E$ and $t_a \bullet = \{e\}$. Since $\text{isLive}(E)$, the transition t_a is the only one enabled under M , so the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of t_a . The marking encoding gives $M(e'') = 1$ for all $e'' \in E$, which is the only reachable case for an and-join before it has fired. Firing t_a yields M'' with $M''(e'') = M(e'') - 1 = 0$ for all $e'' \in E$ and $M''(e) = M(e) + 1 = 1$. Setting $C' = \text{andJoin}(\text{setDead}(E), e, 1)$, by rule **L-AndJoin-NC** we have $C \xrightarrow{\alpha}_L C'$, and $\text{PN}(C') = (N, M')$. By definition, $M'(e'') = 0 = M''(e'')$ for all $e'' \in E$ and $M'(e) = 1 = M''(e)$, so the first condition holds. The second condition holds vacuously: firing t_a affects no dead propagation place, concluding the case.

XorSplit: Let $\text{PN}(\text{xorSplit}(e, \Sigma, E)) = (P, T, F)$ with $P = \{e\} \cup E \cup \{\overline{e''} \mid e'' \in E\}$, $T = \{t_{e''} \mid e'' \in E\}$, and $F = \{(e, t_{e''}), (t_{e''}, e'')\}_{e'' \in E} \cup \{(t_{e''}, e''') \mid e'', e''' \in E, e''' \neq e''\}$, so $\bullet t_{e''} = \{e\}$ and $t_{e''} \bullet = \{e''\} \cup \{\overline{e'''} \mid e''' \in E \setminus \{e''\}\}$ for each $e'' \in E$. The marking encoding gives $M(e) = 1$ and, since e is live, $M(\overline{e''}) = 0$ for all $e'' \in E$, which is the only reachable case for an xor-split before it has fired; hence all transitions $t_{e''}$ are enabled under M . The single firing step $(N, M) \rightarrow (N, M'')$ is the firing of some $t_{e'}$ for a chosen $e' \in E$. Firing $t_{e'}$ yields M'' with $M''(e) = 0$, $M''(e') = 1$, $M''(e'') = 0$ for all $e'' \in E \setminus \{e'\}$, and $M''(\overline{e''}) = 1$ for all $e'' \in E \setminus \{e'\}$. Setting $C' = \text{xorSplit}(e, \mathbf{d}, \{e'.1\} \sqcup \text{setDead}(E \setminus \{e'\}))$, by rule **L-XorSplit-NC** we have $C \xrightarrow{\alpha}_L C'$, and $\text{PN}(C') = (N, M')$. By definition, $M'(e) = 0 = M''(e)$ and $M'(e') = 1 = M''(e')$, so the first condition holds. For the second condition, $M''(\overline{e'}) = 0$, and since e' is live in C' , by safety $M'(\overline{e'}) = 0$, so $M''(\overline{e'}) \geq M'(\overline{e'})$ holds trivially. For all $e'' \in E \setminus \{e'\}$, the Petri net produces $M''(\overline{e''}) = 1$, while $M'(\overline{e''}) \in \{0, 1\}$ depending on whether e'' is a frontier dead edge, hence $M''(\overline{e''}) \geq M'(\overline{e''})$. Together, both conditions of the theorem are satisfied, concluding the case.

XorJoin: Let $\text{PN}(\text{xorJoin}(E, e, \Sigma)) = (P, T, F)$ with $P = E \cup \{e\}$, $T = \{t_{e''} \mid e'' \in E\}$, and $F = \{(e'', t_{e''}), (t_{e''}, e)\}_{e'' \in E}$, so $\bullet t_{e''} = \{e''\}$ and $t_{e''} \bullet = \{e\}$ for each $e'' \in E$. The marking encoding gives $M(e') = 1$ for the live edge e' and $M(e'') = 0$ for all $e'' \in E \setminus \{e'\}$. Since $\bullet t_{e''} = \{e''\}$ for all $e'' \in E$, only $t_{e'}$ is enabled under M , so the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of $t_{e'}$. Firing $t_{e'}$ yields M'' with $M''(e') = 0$, $M''(e'') = 0$ for all $e'' \in E \setminus \{e'\}$, and $M''(e) = 1$. Setting $C' = \text{xorJoin}(\{e'.\mathbf{d}\} \sqcup \text{setDead}(E \setminus \{e'\}), e, 1)$, by rule **L-XorJoin-NC** we have $C \xrightarrow{\alpha}_L C'$, and $\text{PN}(C') = (N, M')$. By definition, $M'(e') = 0 = M''(e')$, $M'(e'') = 0 = M''(e'')$ for all $e'' \in E \setminus \{e'\}$, and $M'(e) = 1 = M''(e)$, so the first condition holds. The second condition holds vacuously: firing $t_{e'}$ affects no dead propagation place, concluding the case.

OrSplit: Let $\text{PN}(\text{orSplit}(e, \Sigma, E)) = (P, T, F)$ and recall that the encoding introduces, for every $S \in \mathcal{P}_\emptyset(E)$, a transition t_S with $\bullet t_S = \{e\}$ and $t_S \bullet = \{e' \mid e' \in S\} \cup \{\overline{e''} \mid e'' \in E \setminus S\}$. The marking encoding gives $M(e) = 1$, and since e is live, $M(e') = 0$ and $M(\overline{e'}) = 0$ for all $e' \in E$ which is the only reachable case for an OR-split before it has fired. Since $M(e) = 1$, all transitions t_S , for $S \in \mathcal{P}_\emptyset(E)$, are enabled under M ; the single firing

step $(N, M) \rightarrow (N, M'')$ is the firing of some transition t_S , for a chosen $S \in \mathcal{P}_\emptyset(E)$. Set $E_1 := S$ and $E_2 := E \setminus S$; since $S \in \mathcal{P}_\emptyset(E)$, $E_1 \neq \emptyset$. Firing t_{E_1} consumes the token from e and produces, in a single step, one token in e' for every $e' \in E_1$, and one token in $\bar{e}''$ for every $e'' \in E_2$. The resulting marking M'' satisfies: $M''(e) = 0$, $M''(e') = 1$ and $M''(\bar{e}') = 0$ for all $e' \in E_1$, $M''(e'') = 0$ and $M''(\bar{e}'') = 1$ for all $e'' \in E_2$. Recall the L-OrSplit-NC rule:

$$\frac{E_1 \neq \emptyset \quad E = \text{setLive}(E_1) \sqcup \text{setDead}(E_2)}{\text{orSplit}(e.1, E_1 \sqcup E_2) \xrightarrow{(\text{d}:\{\bar{e}\} \sqcup E_2, 1:E_1)} \text{orSplit}(e.d, E)}$$

Setting $C' = \text{orSplit}(e.d, \text{setLive}(E_1) \sqcup \text{setDead}(E_2))$, by rule L-OrSplit-NC we have $C \xrightarrow{\alpha}_L C'$, and $\text{PN}(C') = (N, M')$. By definition, $M'(e) = 0 = M''(e)$ and $M'(e') = 1 = M''(e')$ for all $e' \in E_1$, so the first condition holds. For the second condition, for all $e' \in E_1$, $M''(\bar{e}') = 0$, and since e' is live in C' , by safety $M'(\bar{e}') = 0$, so $M''(\bar{e}') \geq M'(\bar{e}')$ holds trivially. For all $e'' \in E_2$, the Petri net produces $M''(\bar{e}'') = 1$, while $M'(\bar{e}'') \in \{0, 1\}$ depending on whether e'' is a frontier dead edge, hence $M''(\bar{e}'') \geq M'(\bar{e}'')$. Together, both conditions of the theorem are satisfied, concluding the case.

OrJoin: Let $\text{PN}(\text{orJoin}(E, e.\Sigma)) = (P, T, F)$ and recall that the encoding introduces, for every $S \in \mathcal{P}_\emptyset(E)$, a transition t_S with $\bullet t_S = \{e' \mid e' \in S\} \cup \{\bar{e}'' \mid e'' \in E \setminus S\}$ and $t_S \bullet = \{e\}$. By definition, the marking encoding gives, for each $e'' \in E$, exactly one of $M(e'') = 1$ or $M(\bar{e}'') = 1$, corresponding to $\neg \text{isWait}(E)$, which is the only reachable case for an OR-join before it has fired. Since $\bullet t_S$ for $S \in \mathcal{P}_\emptyset(E)$ requires a token in e' for every $e' \in S$ and in $\bar{e}''$ for every $e'' \in E \setminus S$, exactly one transition t_S is enabled under M , namely for S equal to the set of currently live edges; the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of this transition t_S . Set $E_1 := S$ and $E_2 := E \setminus S$. We consider here the case $E_1 \neq \emptyset$, as required by rule L-OrJoin-NC below; the case $E_1 = \emptyset$ corresponds to dead-status propagation and is treated in Section D. By definition, $M(e') = 1$ for all $e' \in E_1$, $M(\bar{e}'') = 1$ for all $e'' \in E_2$, and all other places are empty. Recall the L-OrJoin-NC rule:

$$\frac{\neg \text{isWait}(E) \quad \{e'.\Sigma \in E \mid \Sigma = 1\} \neq \emptyset}{\text{orJoin}(E, e.\Sigma) \rightarrow_L \text{orJoin}(\text{setDead}(E), e.1)}$$

Firing t_{E_1} consumes, in a single step, the token from each $e' \in E_1$ and from each $\bar{e}'' \in E_2$, and produces one token in e . The resulting marking M'' satisfies: $M''(e) = 1$, $M''(e'') = 0$ and $M''(\bar{e}'') = 0$ for all $e'' \in E$. Setting $C' = \text{orJoin}(\text{setDead}(E), e.1)$, by rule L-OrJoin-NC we have $C \xrightarrow{\alpha}_L C'$, and $\text{PN}(C') = (N, M')$. By definition, $M'(e) = 1 = M''(e)$, $M'(e'') = 0 = M''(e'')$ for all $e'' \in E$, so the first condition holds. For the second condition, for all $e'' \in E$, the Petri net produces $M''(\bar{e}'') = 0$, and since e is live in C' , by safety $M'(\bar{e}'') = 0$, so $M''(\bar{e}'') \geq M'(\bar{e}'')$ holds trivially. Together, both conditions of the theorem are satisfied, concluding the case.

Composition: The component is $C = (C_1 \mid C_2)$, with $\llbracket C \rrbracket = \llbracket C_1 \rrbracket \oplus \llbracket C_2 \rrbracket$, so $N = N_1 \oplus N_2$ and, by definition of $\llbracket \cdot \rrbracket$ on compositions, $M(p) = M_1(p)$ for

all $p \in P_1$ and $M(p) = M_2(p)$ for all $p \in P_2$, where $(N_1, M_1) = \llbracket C_1 \rrbracket$ and $(N_2, M_2) = \llbracket C_2 \rrbracket$. The transition t fired in the single step $(N, M) \rightarrow (N, M'')$ belongs to either T_1 or T_2 . Without loss of generality, assume $t \in T_1$ (the other case is symmetric, using rule **M-StatusUpd₂**). Since $\bullet t, t\bullet \subseteq P_1$, firing t leaves every place in P_2 unchanged, i.e. $M''(p) = M_2(p)$ for all $p \in P_2$, and restricts to a firing $(N_1, M_1) \rightarrow (N_1, M'_1)$ with $M'_1 = M''|_{P_1}$. By inductive hypothesis applied to C_1 , there exists a reachable marked BPMN term C'_1 such that $C_1 \xrightarrow{\alpha}_L C'_1$ and $\llbracket C'_1 \rrbracket = (N_1, M'_1)$, with $M'_1(e) = M''(e)$ for all $e \in E(C_1)$ and $M'_1(\bar{e}) \geq M''(\bar{e})$ for all $e \in \phi(C_1)$. Recall the **M-StatusUpd₁** rule:

$$\frac{C_1 \xrightarrow{\alpha}_L C'_1}{(C_1 \mid C_2) \xrightarrow{\alpha}_L (C'_1 \mid C_2 \star \alpha)}$$

Setting $C' = (C'_1 \mid C_2 \star \alpha)$, we thus have $C \xrightarrow{\alpha}_L C'$. The firing $(N_1, M_1) \rightarrow (N_1, M'_1)$ together with $C_1 \xrightarrow{\alpha}_L C'_1$ satisfy the hypotheses of Lemma 2, which yields a firing $(N, M) \rightarrow (N, M''')$ such that $M'''(e) = M''(e)$ for all $e \in E(C)$ and $M'''(\bar{e}) \geq M''(\bar{e})$ for all $e \in \phi(C)$, where $(N, M') = \llbracket C' \rrbracket$. Since firing a transition from a given marking is deterministic (Definition 2), and both M'' and M''' arise from firing the same transition t from the same marking M , we have $M''' = M''$. Hence $M''(e) = M'(e)$ for all $e \in E(C)$, so the first condition holds, and $M''(\bar{e}) \geq M'(\bar{e})$ for all $e \in \phi(C)$, so the second condition holds, concluding the case.

Dead Propagation Task: When e is in $\phi(C)$, by definition, the full encoding $\widehat{\text{task}}(e, e')$ also includes the dead propagation net $\overline{\text{PN}(\text{task}(e, e'))}$ with $P = \{\bar{e}, \bar{e}'\}$, $T = \{\bar{t}_t\}$, and $F = \{(\bar{e}, \bar{t}_t), (\bar{t}_t, \bar{e}')\}$, so $\bullet \bar{t}_t = \{\bar{e}\}$ and $\bar{t}_t \bullet = \{\bar{e}'\}$. Recall the **D-Task** rule:

$$\text{task}(e, \mathbf{d}, e', \mathbf{w}) \xrightarrow{\mathbf{d}:e'}_L \text{task}(e, \mathbf{d}, e', \mathbf{d})$$

The marking encoding gives $M(\bar{e}) = 1$ with $\Sigma = \mathbf{d}$, which is the only reachable case triggering dead propagation; hence $\bar{t}_t$ is the only transition enabled under M , so the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of $\bar{t}_t$. Firing $\bar{t}_t$ yields M'' with $M''(\bar{e}) = 0$ and $M''(\bar{e}') = 1$. Setting $C' = \text{task}(e, \mathbf{d}, e', \mathbf{d})$, by rule **D-Task** we have $C \xrightarrow{\alpha}_L C'$, and $\text{PN}(C') = (N, M')$. The first condition holds trivially: no live place is involved in this rule, so $M''(e''') = M'(e''') = 0$ for all $e''' \in E(C)$. For the second condition, $e, e' \in \phi$; the Petri net produces $M''(\bar{e}) = 0$, and since $M'(\bar{e}) = 0$ by definition of C' (the incoming dead status has already been propagated), $M''(\bar{e}) \geq M'(\bar{e})$ holds trivially. For e' , the Petri net produces $M''(\bar{e}') = 1$, while $M'(\bar{e}') \in \{0, 1\}$ depending on whether e' is a frontier dead edge, hence $M''(\bar{e}') \geq M'(\bar{e}')$. Together, both conditions of the theorem are satisfied, concluding the case.

Dead Propagation Split Gateways The dead-propagation rules for all the split gateways have the same premises and effects. Therefore, it is sufficient to prove the result for the XOR-split gateway, as the proofs for the AND-split

and OR-split gateways are analogous. Consider the rule

$$\frac{\neg \text{isDead}(E), \quad \{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset}{\text{xorSplit}(e.d, E) \xrightarrow{\text{d}:E}_L \text{xorSplit}(e.d, \text{setDead}(E))}$$

with $\neg \text{isDead}(E)$ and $\{e'.\Sigma \in E \mid \Sigma = 1\} = \emptyset$. By definition of the dead propagation encoding, $\overline{\text{PN}(\text{xorSplit}(e, E))}$ is an AND-split net on overlined edges, with $P = \{\bar{e}\} \cup \{\bar{e}' \mid e' \in E\}$, $T = \{t_a\}$, and $F = \{(\bar{e}, t_a)\} \cup \{(t_a, \bar{e}') \mid e' \in E\}$, so $\bullet t_a = \{\bar{e}\}$ and $t_a \bullet = \{\bar{e}' \mid e' \in E\}$. Since e is dead, the marking encoding gives $M(\bar{e}) = 1$, which is the only reachable case triggering dead propagation; hence t_a is the only transition enabled under M , so the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of t_a . Firing t_a yields M'' with $M''(\bar{e}) = 0$ and $M''(\bar{e}') = 1$ for all $e' \in E$. Setting $C' = \text{xorSplit}(e.d, \text{setDead}(E))$, by rule D-XorSplit we have $C \xrightarrow{\alpha}_L C'$, and $\text{PN}(C') = (N, M')$. The first condition holds trivially: no live place is involved in this rule, so $M''(e''') = M'(e''') = 0$ for all $e''' \in E(C)$. For the second condition, the Petri net produces $M''(\bar{e}) = 0$, and since $M'(\bar{e}) = 0$ by definition of C' (the incoming dead status of e has already been propagated), we have $M''(\bar{e}) \geq M'(\bar{e})$ trivially. For all $e' \in E$, the Petri net produces $M''(\bar{e}') = 1$, while $M'(\bar{e}') \in \{0, 1\}$ depending on whether e' is a frontier dead edge, hence $M''(\bar{e}') \geq M'(\bar{e}')$. Together, both conditions of the theorem are satisfied, concluding the case.

Dead Propagation Join Gateways The dead-propagation rules for all join gateways have identical premises and effects. Therefore, it is sufficient to prove the result for the XOR-join gateway, as the proofs for the AND-join and OR-join gateways are analogous. Consider the rule

$$\frac{\text{isDead}(E)}{\text{xorJoin}(E, e.w) \xrightarrow{\text{d}:e}_L \text{xorJoin}(E, e.d)}$$

with $\text{isDead}(E)$. By definition, $\overline{\text{PN}(\text{xorJoin}(E, e))}$ is an AND-join net on overlined edges, with $P = \{\bar{e}'' \mid e'' \in E\} \cup \{\bar{e}\}$, $T = \{t_a\}$, and $F = \{(\bar{e}'', t_a) \mid e'' \in E\} \cup \{(t_a, \bar{e})\}$, so $\bullet t_a = \{\bar{e}'' \mid e'' \in E\}$ and $t_a \bullet = \{\bar{e}\}$. Since $\text{isDead}(E)$, the marking encoding gives $M(\bar{e}'') = 1$ for all $e'' \in E$, which is the only reachable case triggering dead propagation; hence t_a is the only transition enabled under M , so the single firing step $(N, M) \rightarrow (N, M'')$ must be the firing of t_a . Firing t_a yields M'' with $M''(\bar{e}'') = 0$ for all $e'' \in E$ and $M''(\bar{e}) = 1$. Setting $C' = \text{xorJoin}(E, e.d)$, by rule D-XorJoin we have $C \xrightarrow{\alpha}_L C'$, and $\text{PN}(C') = (N, M')$. The first condition holds trivially: no live place is involved in this rule, so $M''(e''') = M'(e''') = 0$ for all $e''' \in E(C)$. For the second condition, for all $e'' \in E$, the Petri net produces $M''(\bar{e}'') = 0$, and since $M'(\bar{e}'') = 0$ by definition of C' (the incoming dead status of e'' has already been propagated), we have $M''(\bar{e}'') \geq M'(\bar{e}'')$ trivially. For e , the Petri net produces $M''(\bar{e}) = 1$, while $M'(\bar{e}) \in \{0, 1\}$ depending on whether e is a frontier dead edge, hence $M''(\bar{e}) \geq M'(\bar{e})$. Together, both conditions of the theorem are satisfied, concluding the case. $\square$